UniRL: Trajectory-Centric Post-Training Across Heterogeneous Generative Models

UniRL Authors *authors@to.be.added*
Affiliations to be added before the arXiv release

Abstract

Reinforcement-learning systems share an outer loop—generate, score, assign credit, and update—yet their implementations fragment because autoregressive tokens, diffusion trajectories, multimodal values, and multi-stage rollouts carry different state and impose different execution constraints. We present UniRL, a trajectory-centric system that places a lineage-preserving, modality-aware intermediate representation at the boundary between post-training semantics and distributed execution. A trajectory batch is a sequence of typed layers, one per depth of a forest of prompt-rooted trees, carrying decoded values, model conditions, replay segments, rewards, and behavior-policy versions. A rollout request is a skeleton of such layers whose generated layers are still empty shells, and every engine fills them through one `Sample` → `Sample` transformation. The same trajectory can carry autoregressive and diffusion stages linked by lineage, while logical roles, placement, tensor transport, weight publication, and memory residency determine how the fill executes. These contracts factor model-specific replay from lineage, movement, placement, and scheduling decisions; they do not assert that the underlying objectives or every engine–topology combination are interchangeable. The design is grounded in a source audit of UniRL release v0.2.0 (commit `fd344ad`); subsequent experiment reports provide initial GPU evidence. Three-seed diffusion results show improved held-out automatic scores together with reduced within-prompt diversity, while an eight-GPU native-recipe comparison reports lower iteration time than VeRL-Omni. A separate, single-run, recipe-matched 500-step Qwen3-4B GRPO comparison records 8.2% higher end-to-end output-token throughput for UniRL than veRL while generating 11.0% longer responses. A source-importing CPU harness exercises ten core IR invariants. The present evaluation distinguishes legacy report-derived measurements from the newly regenerated AR systems evidence: the primary AR quality endpoint, real-rollout lineage validation, paired representation cost, and a controlled time-to-quality result remain open.

1 Introduction

Online post-training is a feedback program. It expands inputs into policy samples, evaluates those samples, turns feedback into a learning signal, and applies one or more optimizer updates. At this level, language-model GRPO, diffusion-model policy gradients, prompt-rewrite pipelines, and tool-using agents look deceptively similar. Their data paths do not. An autoregressive update may need packed token IDs, masks, rollout log probabilities, and per-token values; a diffusion update may need a sparse set of latent states, a noise schedule, and transition statistics; a composed workflow must retain which image descended from which rewritten prompt; an asynchronous run must additionally retain which policy version generated every sample.

Existing systems have made major progress on scalable LLM post-training and flexible device mapping (Sheng et al., 2024; Hu et al., 2024; Wang et al., 2025; NVIDIA Corporation, 2025). Fully or partially asynchronous systems address rollout tail latency and accelerator idle time (Fu et al., 2025; Zhang et al., 2026; Zhu et al., 2025). Meanwhile, diffusion RL methods and multimodal frameworks broaden the optimization and model families that can be trained (Black et al., 2023; Liu et al., 2025; Xue et al., 2025; Zheng et al., 2025; Huang

et al., 2026). The remaining design question is how the information shared by these workflows should survive changes in their model-specific state and execution paths.

> *Can one post-training program cross generative model families and execution topologies while retaining the structure needed for correct grouping, replay, credit assignment, transport, and policy-version control?*

Its answer is to make a trajectory intermediate representation (IR) the stable semantic object, then expose execution choices as explicit contracts around that object. The IR makes a complete prompt-rooted lineage, rather than an individual row or an LLM-specific tensor dictionary, the unit of scheduling and sharding. Typed leaves distinguish decoded multimodal values from model conditions and optimization segments. Distributed methods declare how each structured field scatters, concatenates, packs, or remains shared. Dense tensors can be replaced by row-addressable references without changing the trajectory structure. This representation is passed through a small set of logical roles and can be mapped to colocated or disjoint GPU slabs.

The paper develops two coupled design ideas:

- A *lineage-preserving trajectory IR* in which a rollout request is a typed tree skeleton that can span autoregressive and diffusion stages, each generated layer is conditioned through its ancestry and keeps its own replay segment, and training folds reward and credit back along the same tree (Section 3). Synthetic equality checks provide initial semantic evidence; recorded-rollout validation and the primary AR quality endpoint remain open alongside the reported diffusion results.
- A *factorization of semantic roles from physical execution*: rollout remains one `Sample` → `Sample` fill of that skeleton, while field-aware dispatch, tensor references, placement, weight publication, and residency specify its execution (Section 4). Native-recipe, recipe-matched AR, and internal topology comparisons provide initial performance observations; the paired representation-cost control remains unmeasured.

Asynchronous execution is where the second idea pays off most directly. Because rollout, reward, and training are logical roles rather than fixed steps of one loop, the orchestration layer—admission, root-complete grouping, version stamping, lag filtering, and weight publication—can be lifted out of individual trainers. UniRL's AR and diffusion trainers therefore share one bounded-staleness manager over the same trajectory IR (Section 4.5), and we exercise it on both workloads. For AR, single-seed capacity measurements with overlap enabled show lower per-rollout time than synchronous colocated rollout; the accompanying quality pilot did not pass, so no time-to-quality benefit is claimed (Section 8.4). For diffusion, denoising and replay compute dominate each iteration and leave little generation idle time for overlap to recover, so asynchrony yields no significant gain. **TODO[data]:** The diffusion asynchronous measurements are not yet in the evidence bundle; add them to Section 8.4 before release.

The evidence is organized from structural correctness, through credible learning and a composed workflow, to measured system cost. The experiment IDs and corresponding artifacts are specified in Section 8.

The current evidence comprises an archived CPU regression, reported GPU results for diffusion training, native-recipe systems performance and deployment topology, and a newly regenerated recipe-matched AR systems comparison. The diffusion result includes a material diversity regression; the diffusion systems comparison retains estimator and trainable-parameter differences, while the AR comparison contains only one process-level run per framework. These observations do not establish a cross-workload or time-to-quality advantage or complete the trajectory-centric claim: real-rollout semantic validation and a paired measurement of representation cost remain necessary.

2 Problem: Preserve Semantics While Changing Execution

Let an online post-training iteration consume a batch of roots X, sample trajectories with behavior policy π_v, obtain reward r, construct credit A, and update parameters θ:

$$X \xrightarrow{\pi_v} \mathcal{T} \xrightarrow{R} (\mathcal{T}, r) \xrightarrow{C} (\mathcal{T}, A) \xrightarrow{U_\theta} \theta'. \tag{1}$$

The compact notation hides three forms of heterogeneity.

Trajectory heterogeneity. The sufficient statistics for an update depend on the generative process. For AR policies, a segment may be a variable-length packed token sequence with masks, old log probabilities, values, and returns. For stochastic diffusion or flow policies, a segment may contain selected latent snapshots, a shared noise schedule, the indices of sampled transitions, and rollout or replay transition statistics. Multi-stage and agentic paths add nonuniform lineage and observations.

For example, let prompt p produce rewrites $w_{p,j}$ and images $x_{p,j,k}$. The image model conditions on $w_{p,j}$, but credit may compare either images under that rewrite or all images under p. A flattened image batch can implement both choices only if it retains or reconstructs the relevant parent keys. This example motivates explicit ancestry, while the image model's replay state motivates typed segments; neither requirement follows from the other.

Role heterogeneity. Generation, reward evaluation, replay, gradient computation, and policy publication differ in runtime, parallel layout, and lifecycle, not only in kernels. Rollout may run inside the training process or in a dedicated serving engine with its own tensor-parallel layout, such as SGLang (Zheng et al., 2024), vLLM (Kwon et al., 2023), or vLLM-Omni (Yin et al., 2026), while a sharded training backend such as FSDP (Zhao et al., 2023) or VeOmni (Ma et al., 2025a) owns parameters and optimizer state. Reward ranges from an in-process scorer to an environment-isolated scorer process or a remote multi-model service. These roles must agree on when each copy of the policy is resident, offloaded, or published, and publishing from a sharded trainer is itself a collective over the whole training mesh. Which boundary dominates cost depends on the configuration: in the measured separate-slab deployment, the report attributes much of the slowdown to reward processing (Section 8.3).

Topology heterogeneity. A small run may time-share one GPU set between training and rollout; a larger run may separate them; a variable-duration rollout may benefit from asynchronous admission. Each choice changes memory pressure, transfer paths, synchronization frequency, and allowable policy lag.

These observations produce four requirements (Table 1). The first two follow from trajectory heterogeneity and the third from role and topology heterogeneity; the fourth specializes the third to asynchronous execution, where decoupled roles may train on work generated by an older policy.

Table 1: Design requirements and the corresponding UniRL mechanism. The right column states what must be evaluated; it is not a result.

Requirement	Mechanism	Evidence required
Preserve lineage	ID-encoded ancestry validated at construction; split, select, concat, data-parallel sharding, and reward propagation over complete root groups	Trace invariants across AR, diffusion, and cross-stage runs against an ID-joined reference
Preserve replay inputs	Typed primitives and conditions; packed AR and sparse latent replay segments; σ-schedule echo checks on external diffusion engines	Rollout–replay consistency on exercised model paths
Change execution safely	Logical roles, placement scopes, field-aware dispatch, tensor references, weight publication and residency lifecycle, and startup rejection of unvalidated combinations	Replay parity after each publication path; matched-work topology and transport ablations
Control policy age	Per-part behavior versions, complete-group admission, lag filters, quiescent publication, mixed-version rejection, and hard boundaries	Lag distributions, throughput, and time-to-quality

Non-goals. UniRL does not erase model-specific code; it confines it to a model bundle—architecture-specific stages, conditions, and rollout adapters—and to an algorithm that defines its loss over replay. It

does not automatically discover an optimal placement. The present runtime uses a driver to sequence roles and Ray actors (Moritz et al., 2018) to host them; it is not a new cluster scheduler.

3 A Trajectory-Centric Programming Model

3.1 A trajectory batch is a sequence of tree layers

The central type is

$$\mathcal{T} = [P_0, P_1, \ldots, P_L], \qquad P_i = (I_i, X_i, C_i, Z_i, R_i, A_i, V_i, M_i), \tag{2}$$

where I_i are sample identifiers, X_i decoded modality values, C_i model conditions, Z_i optimization segments, R_i rewards, A_i advantages or returns, V_i the generating engine's policy version (behavior-version provenance only on the asynchronous paths; Section 4.3), and M_i auxiliary metadata. In code, $\mathcal{T}$ is a `Sample` and each P_i is a batched `Part`, rather than a single tree node. The sequence encodes a forest of root-prompt trees: P_i holds every node at depth i, and edges connect adjacent layers. We call the last layer the *frontier*; trajectories grow by appending layers to it. A generated layer is typed by the sampling-parameter class that produced it and the segment class it carries, for example an AR layer with a `TextSegment` or a diffusion layer with a `LatentSegment`; input layers carry the prompts and observations that condition generation.

Identifiers encode ancestry. Root IDs contain no lineage separator; a branch appends a child ordinal. For every non-root row j in P_i, its parent ID must occur in P_{i-1}:

$$\forall i > 0,\ \forall s \in I_i : \quad \text{parent}(s) \in I_{i-1}. \tag{3}$$

The constructor checks Equation 3. The `fork` operator appends k generated children per row of the frontier; `observe` appends a branch-one input from an environment; and `propagate_rewards` reduces child reward to an unscored parent.

Batch operations act on complete root groups. `Sample.split()` returns one tree-complete trajectory per root, and slicing or selection is implemented by selecting these groups rather than independent rows. Moving complete roots preserves the parent-contiguous order constructed by `fork`; independently permuting child rows need not preserve the credit computation.

Well-formed batches. Lineage operations are defined over well-formed trajectory batches: root IDs are unique, trajectories being merged have the same number of parts, and siblings follow the parent-major order produced by `fork`. The programming model establishes these properties by construction—trajectories grow only through `fork` and `observe`, and batches are sharded and selected as complete root groups—so credit assignment and reward reduction can consume each sibling group as one contiguous block. Group-advantage computation rejects batches whose siblings are not contiguous, part concatenation requires matching decoded-value metadata across shards, and the asynchronous batch assembler admits only batches generated by a single behavior version. E0 is the gate that checks these properties at the actual experiment boundaries (Appendix D).

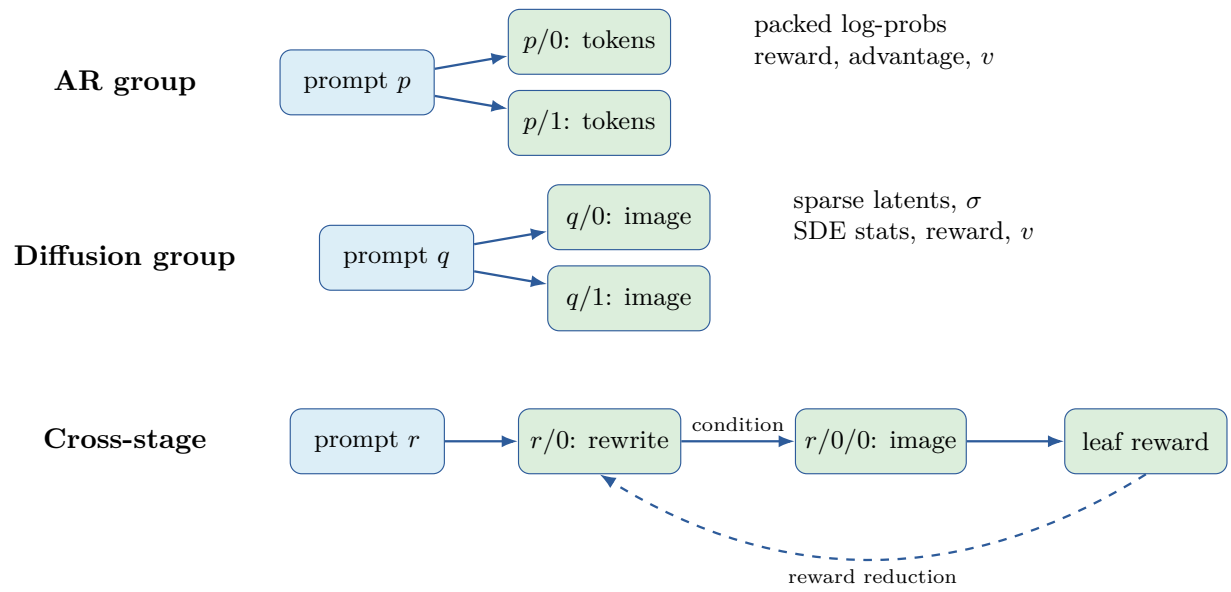


Figure 1: Three trajectory shapes represented by the same ordered-part IR. A generated part retains decoded values and the optimization segment required to replay its policy. The policy version v is attached to generated layers. The cross-stage path illustrates available ancestry; E4 freezes the rewrite model and trains only diffusion.

3.2 Values, conditions, and segments are separate layers

UniRL distinguishes three kinds of leaves that are often conflated.

Primitives. Primitives are decoded or externally supplied values: text, images, videos, audio waveforms, and media references. Image pixels and variable-duration video/audio values use packed representations with per-example shapes or cumulative offsets. They are suitable for reward functions, visualization, and subsequent turns, but need not be the tensors replayed by the trainable model.

Conditions. Conditions are model-facing encodings, such as text embeddings or image latents. They allow rollout adapters and train-side stages to agree on typed inputs without prescribing one tokenizer, text encoder, or multimodal fusion architecture.

Segments. Segments retain optimization-facing state. `TextSegment` stores variable-length packed token fields. `LatentSegment` stores latent snapshots at selected indices, a shared sigma schedule, and optional SDE log probabilities or transition means. The sparse index map is important: a diffusion algorithm can replay only selected transitions instead of materializing an entire denoising path for every loss.

The distinction makes decoded payload lifetime independent of replay lifetime. After reward logging, trainers may drop large decoded media while retaining conditions, segments, reward, and credit for training.

3.3 A field algebra defines structural batching

Every structured batch field carries a kind: concatenated, packed variable-length, or shared (the default for an unannotated field); scalar reductions by maximum, minimum, sum, or mean are also available but are applied only when concatenating, and the current source uses only the maximum, for timing metadata. Concatenation, row selection, slicing, and repetition act recursively on concatenated and packed fields and pass shared fields through unchanged; device movement applies to every tensor regardless of kind. The same kinds define how a distributed method reconstructs its return value. For example, a DP-scattered `Sample` is split by root groups (the root count must be divisible by the data-parallel size), and the data-parallel

heads' results are concatenated position-wise; another argument whose leading size differs from the split size is replicated to every shard rather than rejected. A broadcast lifecycle method returns the list of per-worker results, and callers that expect a replicated value check agreement explicitly.

This removes hand-authored tensor slicing from each role; merging shards relies on the well-formed-batch properties above. It also creates a measurable cost: recursive structural operations and reference bookkeeping must be benchmarked in Section 8.3 rather than assumed free.

3.4 Rollout unfolds a declared trajectory tree

A rollout request is the skeleton of the tree it will become. Starting from the root layer, the caller appends one generated layer per stage with `fork`; until an engine fills it, such a layer is a *shell*. Each shell fixes its child count, carries child identifiers that already name their parents, and holds a typed sampling-parameter object—autoregressive or diffusion—that identifies the generative process that must fill it. The prompt-enhancement and unified-model trainers build

$$\texttt{request}(P_0).\texttt{fork}(N, s^{\mathrm{AR}}).\texttt{fork}(M, s^{\mathrm{diff}}) = [P_0,\ P_1^{\mathrm{AR}},\ P_2^{\mathrm{diff}}], \tag{4}$$

where s^{AR} and s^{diff} are the stages' sampling parameters, and whose BN AR rows and BNM image rows, for B root prompts, are fixed before any token or latent is sampled. Every rollout engine implements one transformation over such a skeleton:

$$F_{\theta,e} : \mathcal{T}_{\mathrm{skeleton}} \longrightarrow \mathcal{T}_{\mathrm{filled}}, \tag{5}$$

where e identifies a train-side or external generation runtime. F keeps the request's part sequence and identifiers and writes decoded values, conditions, and a replay segment into the shells it fills, and stamps its policy version on the frontier it returns; when an engine or trainer fills several shells through child engines, it copies each child's version onto the shell that child filled. The base contract also exposes lifecycle and policy-publication operations, but it does not standardize the engine's internal serving API.

One trajectory spans autoregressive and diffusion stages. The filled skeleton records every stage of a multi-stage generation in the trajectory itself. Within a pipeline that fills several shells, a generated shell obtains its inputs from its ancestry: `conditioning_at` walks each row's parent chain and collects the role-tagged primitives along it. In the HunyuanImage-3 and BAGEL text-to-text-and-image modes, one pipeline fills the AR shell from the root prompt; the image shell's lineage then surfaces both the original prompt and the generated reasoning as its conditioning, without a side table that pairs each image with the text that produced it. Each stage's replay state lands in its own layer—a `TextSegment` from an AR stage and a `LatentSegment` from a diffusion stage—so the filled tree carries what credit assignment and replay need for both stages (Section 3.5). An AR stage and a diffusion stage are thus two typed layers of one prompt-rooted tree rather than two batches joined after the fact. The representation carries this cross-stage dataflow; stage order is declared by each workflow's skeleton. The current trainers instantiate single-stage AR, single-stage diffusion, input–AR–diffusion, and multi-turn tool trajectories from the same representation.

Engine forms. The current source contains three direct forms of Equation 5 and one coordinator that composes it. The train-side adapter invokes the model pipeline directly under evaluation mode, filling every generated shell of the request. Dedicated serving adapters for SGLang, SGLang-Diffusion, vLLM, vLLM-Omni, and FastVideo translate typed trajectory inputs to their native requests and rebuild typed responses. A composed engine serially fills the AR and diffusion shells of Equation 4 with two child engines. An agentic harness composes Equation 5 over turns: it forks a policy turn, has a named engine fill it, and appends the environment observation before the next turn, returning a longer part sequence. The claim is therefore about a common boundary, not one universal generation kernel.

3.5 Training folds the tree into per-stage updates

Training traverses the filled tree in the opposite direction. The reward role scores the frontier, which in the cross-stage layouts above is the image layer. `propagate_rewards` then reduces each scored layer onto its unscored parent layer by mean, maximum, or sum, so an upstream AR layer receives feedback from the

images it conditioned. Credit is computed per trainable layer: `compute_advantages` labels each row by its ancestor at a chosen depth—the immediate parent by default, or the root prompt when requested—and, by default, normalizes rewards within each group. For the image layer of Equation 4, the two choices compare images under one rewrite or all images under one original prompt; selecting between them is a lineage query rather than a reconstructed join (Section 8.3). Reduction and grouping consume siblings as contiguous blocks, the well-formedness property stated above. Advantages are attached on the driver before the batch is scattered, so training shards receive precomputed credit rather than re-deriving groups.

Each trainable layer then becomes a training track bound to a train stack and a stage algorithm. The trainer selects the layer by its sampling-parameter type and skips frozen layers. In the composed prompt-enhancement workflow, the AR and diffusion models have separate stacks and optimizer steps; freezing the rewrite model, as E4 does, trains only the diffusion track. When both layers belong to one backbone, as in HunyuanImage-3, a unified stack back-propagates the AR and image losses through one shared optimizer step.

3.6 The algorithm boundary preserves specialized replay

Within a track, the algorithm consumes its layer's conditions, segment, and advantages. A train stack owns microbatch planning, gradient accumulation, global loss normalization, gradient clipping, optimizer steps, and per-update aggregation. An algorithm object owns the stage-specific map

$$(C, Z, A; \theta) \longmapsto \mathcal{L} \quad \text{and backward.} \tag{6}$$

The loss itself sees only its layer's conditions, segment, and advantages; an optional per-shard preparation hook may read the layer's whole part (for example, per-row metadata or critic values), and some algorithms additionally hold a backend handle for reference or EMA weights. Before repeated optimizer updates, the stack can freeze an old-policy anchor at the same microbatch geometry used for training. AR objectives replay token log probabilities; diffusion objectives replay selected transitions and may compare rollout statistics, a pre-update replay, or a reference model. Supervised and teacher-distillation objectives explicitly opt out of advantages; the forward-process DiffusionNFT objective still consumes group advantages.

Because models and algorithms meet only at this data boundary, they compose without pairwise adaptation. A model is integrated once, as a bundle whose stages fill and replay typed segments; an algorithm is written once, as a loss over a stage's replay of those segments. A shared algorithm can then be paired with any model whose stages produce and replay the segment fields it consumes—AR objectives with `TextSegment` producers and diffusion objectives with `LatentSegment` producers—so supporting M models and N shared algorithms requires $M + N$ integrations rather than $M \times N$; a model that needs a specialized objective adds it as a subclass without changing the others. Variants that keep the FlowGRPO loss and change only the SDE transition or the step schedule, such as DanceGRPO and MixGRPO, are recipes rather than new algorithm classes.

This division is deliberately not advertised as algorithmic novelty. GRPO follows the group-relative formulation introduced for language reasoning (Shao et al., 2024); PPO follows proximal policy optimization (Schulman et al., 2017); diffusion objectives build on prior diffusion-RL methods (Black et al., 2023; Liu et al., 2025; Xue et al., 2025; Zheng et al., 2025). The systems question is whether these objectives can consume the same trajectory and execution contracts without hiding their different sufficient statistics.

4 From a Program to a Distributed Execution

Figure 2 separates the feedback program from the mechanisms that execute it.

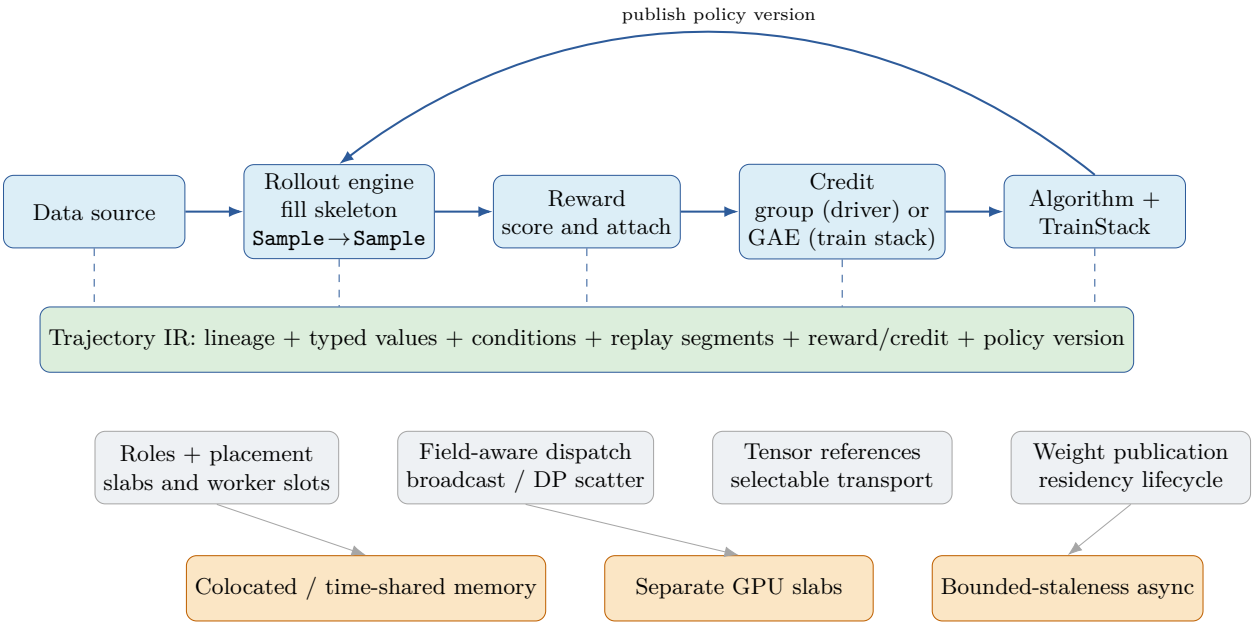


Figure 2: Source-derived UniRL architecture. The top row is the feedback program; the green bar is its stable data boundary. The lower rows map logical roles to communication and resource policies. Not every topology is valid for every engine; startup validation rejects unsupported combinations.

4.1 Logical roles, device slabs, and worker slots

A `DevicePool` creates one Ray worker per GPU initially and reserves one strict-packed placement group per node. Each worker occupies a slot on its GPU: slot 0 exists from startup, further slots are created on demand up to a per-GPU limit, and every slot reserves an equal fraction of that GPU. A top-level placement scope claims a fraction of the whole device pool from the still-unclaimed devices; a nested scope takes a fraction of its parent's devices. Roles created inside a scope either share its worker slot—enabling local sibling access and memory time-sharing—or receive isolated slots on the same devices (isolated slots require the per-GPU store transport). Disjoint top-level scopes form separate training, rollout, or reward slabs.

A role is a regular Python object derived from `Remote`. The handle constructs one role replica on each selected worker and derives data-, tensor-, pipeline-, sequence-, and expert-parallel rank metadata from the role configuration. Method decorators declare a dispatch mode (broadcast, scatter, or data-parallel scatter) and which ranks execute; the dispatch mode determines how results are collected. The trainer is consequently a single-controller program, while each method may invoke multi-controller distributed training or serving inside a role. This resembles HybridFlow's separation between orchestration and distributed computation (Sheng et al., 2024); UniRL's paper claim centers instead on carrying the same typed lineage through heterogeneous generative stages.

4.2 Tensor references decouple structure from movement

In the single-controller design, every role method is a remote call issued by the driver, and its result returns to the driver before being dispatched to the next role. If trajectories carried dense tensors across these hops, latent snapshots, token log probabilities, and decoded media would pass through the driver on every call, and the driver would have to materialize them merely to split, select, or scatter complete root groups. UniRL therefore routes control through the driver but keeps tensor data on the workers. When a role method returns, its worker stores every dense tensor leaf in a local transport and *dehydrates* it into a `TensorRef`: an ordered list of half-open row spans over opaque handles, plus shape, dtype, and device metadata. The driver receives only this dehydrated trajectory; selection and contiguous slicing construct new span views

without moving tensor data. Before the next call, the transport localizes each reference for its destination worker, leaving resident spans in place and transferring the others worker-to-worker. The destination worker then *hydrates* all referenced spans in one batch and writes the tensors back into the nested object in place before invoking the method; spans whose trailing widths differ are zero-padded to a common width. Control decisions on the driver need only small fields such as rewards, which it hydrates on demand.

The transport interface has a deliberately small contract: store tensors, resolve handles, identify references, and optionally localize references for destination workers. The audited source provides (i) a colocated per-worker store, (ii) a per-GPU store with IPC and NCCL transfers, and (iii) a TransferQueue path backed by simple storage or Mooncake-compatible storage. Mooncake itself was proposed for disaggregated serving (Qin et al., 2024); here its storage interface is a transport option rather than a claimed UniRL invention.

The driver can therefore manipulate trajectory structure and metadata without materializing all GPU tensors. Whether this reduces memory and transfer cost in practice is RQ3, not an assumption.

4.3 Residency lifecycles and policy publication

Training and generation often hold different copies or formats of a policy. In every deployment, a due publication completes before the generation it serves, and the engine stamps each generated frontier with its current version. Trainers that publish through the rollout manager set that version to the optimizer version (Section 4.5); the remaining cases are described below. How publication interleaves with memory residency depends on whether the two roles share devices.

Colocated slabs. When training and rollout time-share one GPU set, the engine is put to sleep—releasing its weight and cache memory—between generations and woken before the next one, and training state may be offloaded to host memory while the engine is awake. Every colocated order keeps two invariants: a due publication completes before generation, and memory is released before the role that needs it is brought up, so the engine sleeps before offloaded training state is restored. The default order wakes the engine, publishes if due, offloads training state if enabled, and generates; afterwards it sleeps the engine and restores the training state. The AR and agentic trainers follow this order, and the AR trainer adapts it to engine capabilities: an engine that can restore its weight memory separately from its cache receives weights before allocating the rest, and an engine pinned to one GPU of the slab receives weights that are extracted before training state is offloaded and pushed after the engine wakes (Appendix B). The diffusion trainers express residency as one resident-or-offloaded choice per role (training, rollout, reward): they offload every non-resident role before waking the engine, publish into the awake engine, sleep it after generation unless the rollout role is resident, and restore an offloaded trainer only before its optimizer step. The prompt-enhancement trainer follows the default order, and the unified-model trainer with separate AR and diffusion engines uses the same extract-then-push order. A train-side engine reads live model objects, so training state is never offloaded for it.

Separate slabs. When training and rollout occupy disjoint devices, generation needs no memory handoff:

1. at startup, connect the training side to the rollout workers through a one-time handshake;
2. at each publication point, transfer parameters across slabs, under the admission discipline of Section 4.5 for asynchronous trainers;
3. generate and stamp the frontier with the engine's current version.

Generation never offloads training state to make room for the engine. The synchronous diffusion trainer sleeps the engine between rollouts unless the rollout role is resident and, when the training role is not resident, offloads training state while a reward role shares the training slab. The asynchronous trainers keep the engine awake; the asynchronous diffusion trainer also requires a resident reward role.

Weight-sync implementations cover in-process and cross-slab LoRA adapter pushes and full-weight paths via serialized tensors, CUDA IPC (ZMQ or vLLM's native weight-transfer engine), SGLang checkpoint-engine

IPC, NCCL broadcast, and a checkpoint-file handoff. The base engine can also receive per-track updates for composed models.

The stamped policy version is behavior-version provenance only on the asynchronous paths. The asynchronous and agentic trainers set it to the optimizer-update count at each publication. The synchronous trainers do not advance it, so their frontiers keep version zero, except that the vLLM engine stamps the number of weight transfers it has received through vLLM's native weight-transfer engine.

The source rejects combinations whose lifecycle has not been validated, such as a train-side rollout on a physically separate slab or selected EMA/offload combinations. This fail-closed policy is important because a partially awakened engine or silently stale adapter can produce plausible but invalid training data. Section 8 therefore includes replay-consistency and failure-injection checks alongside throughput.

4.4 Synchronous execution

The synchronous AR trainer performs rollout, reward, credit assignment, replay/update, optional evaluation, and checkpointing in order. The diffusion trainer can accumulate multiple rollouts into one optimizer window and can place reward on the train slab or a third slab, or score micro-batches inside the colocated rollout window so that reward overlaps generation. Topology is a placement choice made through the same scopes for every modality. The synchronous AR trainer builds one colocated slab, optionally pinning the engine to one of its GPUs, and the synchronous diffusion trainer additionally builds separate training and rollout slabs. Separate-slab AR without overlap is the asynchronous trainer with one prompt batch in flight, publication after every batch, and zero allowed lag (Section 4.5): it admits the next batch only after publishing the weights produced from the previous one, so generation and training strictly alternate. In every case, topology changes resource ownership, residency, and the publication path, but not the trajectory boundary: the same `Sample` passes from rollout through reward and credit to the train stack. This makes a controlled topology experiment possible: hold model, prompts, group size, reward, number of replayed steps or tokens, and optimizer updates constant while changing only engine or placement. The publication path and training data-parallel width change with the placement and are reported as part of it.

4.5 Bounded-staleness execution

Asynchrony changes correctness conditions, not just control flow. UniRL's asynchronous AR and diffusion trainers run on separate training and rollout slabs and share one manager. The driver submits whole prompt batches only while they fit the in-flight capacity, the freshness horizon implied by the staleness allowance, and the batches remaining before the next evaluation, checkpoint, or final boundary. A background dispatcher then fills every engine slot up to a configured per-worker capacity. Completed trajectories are grouped by root; only complete groups enter the ready buffer. Every generated part must carry an output policy version v_o.

Let v_t be the current optimizer-update version, which counts committed optimizer steps, and u the configured number of optimizer updates per consumed rollout batch. The manager exposes lag in update units,

$$\ell = v_t - v_o \leq ub, \tag{7}$$

where b is the staleness allowance in rollout-batch units (configurable for AR, fixed to the publication interval minus one for diffusion). The filter measures each trajectory by its oldest generated part and discards whole root groups beyond that budget, including incomplete roots held across a publication. Because a skipped non-finite step does not advance v_t, admitted work can be more than b rollout batches old. Before publishing version v_t, it pauses admission, drains or suspends work, completes any started or batch-aligning prefix, checks that no publication splits a training batch across behavior versions, publishes weights, updates the rollout version, and then resubmits eligible pristine carry. Evaluation and checkpointing require a fully empty manager.

This discipline differs from claiming fully asynchronous optimization. Weight publication remains a barrier. The asynchronous diffusion trainer admits one prompt batch at a time and requires each training batch to come from a single generation request, so its next generation overlaps only the optimizer step; the AR trainer

may keep several batches in flight and refills before reward scoring. The evaluation must report the actual lag histogram, dropped work, and barrier time; a nominal "async" label is insufficient.

5 Reward Serving and Generation–Reward Overlap

Reward is the one role of the feedback program whose cost the policy does not determine. A contrastive preference model scores a batch in well under a second; a vision–language judge with tens of billions of parameters can take as long as the generation it evaluates, needs a different software stack from the trainer, and may not fit beside the policy on one accelerator. UniRL therefore fixes what a reward *is* at the trajectory boundary and leaves where it runs, in which environment, and when it is called relative to generation as separate execution choices.

Source scope. This section describes UniRL commit `651490e` (2026-09-30), which is later than the design-audit commit used elsewhere in the paper. The shared-GPU mechanism of Section 5.5 is an open pull request (#492 at `8bb4119`) and is not part of that commit. The measurements in Section 5.4 lie outside the E0–E6 programme; their per-rollout records and generator are under `artifacts/reported_results/2026-09-30-reward-service/`.

5.1 One scoring contract over many scorers

A trainer calls one method, `score_and_attach`, on the reward role. The call is scattered over tree-complete shards like any other distributed method (Section 4); on each shard it assembles a typed request from the trajectory, asks exactly one backend for scores, and returns a new trajectory whose frontier carries a scalar reward and named component rewards per row. The request holds the frontier's decoded values, the non-text values of earlier turns as conditioning (for example the source image of an edit), the original and the generation prompt, and the sample and group identifiers. A backend states which prompt it scores against; after a rewrite stage the two differ, and the choice decides whether the reward measures fidelity to the user's request or to the rewritten text.

The role refuses to be a silent source of bad credit. It rejects a frontier that already carries rewards, raises when any row fails instead of substituting zero—remote backends count a non-finite score as a failure—and sets the reward of a free-length autoregressive trace that ran into its token limit to zero, or applies a length penalty, as configured. It also rejects a training reward that gives prompt-to-video alignment no weight, since an audio–video agreement score alone can be raised by collapsing both streams; such a score remains available for evaluation. A group-normalized advantage spreads one corrupted reward over every sibling in its group, which is why these cases stop the step.

Table 2 lists what currently implements the contract: 19 in-process backends and 13 scorers registered in the standalone reward service. They cover contrastive and preference models such as CLIP, PickScore, HPS and ImageReward (Radford et al., 2021; Kirstain et al., 2023; Wu et al., 2023; Ma et al., 2025b; Xu et al., 2023), vision–language judges for composition, rendered text and instruction-guided editing (Kamath et al., 2025; Wu et al., 2025; Luo et al., 2025), video and audio–video alignment models, and text graders for reasoning workloads. A remote backend can request a panel of scorers for each row and reduce it by a weighted sum, mean, minimum or maximum; the individual scores stay attached as components for logging. Presence in the table means that the code path exists; it is not evidence that every scorer has been exercised in training.

Table 2: Reward implementations at commit `651490e`, by the output they score. Names are registry keys. *Registered but disabled in the example service configuration.

Output scored	In-process backends	Reward-service scorers
Image	`clip`, `pickscore`, `hpsv2`, `hpsv3`, `hpsv3pp`, `image_reward`, `ocr`, `geneval2`	`clip`, `pickscore`, `imagereward`, `hpsv2`, `hpsv3`, `unified_reward`, `geneval2`, `geneval`*, `ocr`, `wise`
Image editing	–	`editreward`, `editscore`
Video	`videopickscore`, `videoclipdelta`, `videoalign`, `video`	`videoalign`*
Audio	`clap`	–
Audio–video	`imagebind`, `t2av_composite`	–
Text	`math_verify`, `mc_exact_match`, `llm_judge`	–
Routing	`per_domain`	–

5.2 Where a scorer runs

The same contract is served from four places, which differ in what they isolate.

In the policy's workers. An in-process backend loads the scorer in the worker that also hosts training and rollout roles. Its memory is governed by the residency policy of Section 4.3: one choice per role states whether the trainer, the rollout engine and the reward keep their weights on the accelerator while idle, and a planner issues only the transitions that change that state. A reward phase that follows generation therefore inherits an already parked trainer instead of moving it again.

On a reward slab. A placement fraction carves the tail of the device pool into a disjoint slab for the reward role. Scoring is scattered over that slab's workers and the decoded values reach it as tensor references, so a heavier scorer stops competing with the policy for memory while the job remains one program.

In a managed child process. Scorers built on a serving engine or on pinned library versions often cannot be imported into the trainer's environment. A managed backend starts one scorer process per reward worker with its own Python interpreter, hands it a listening loopback socket and that worker's single visible device, and speaks HTTP to it. The protocol is deliberately strict. Requests are sent in bounded chunks independent of the shard size. Every item carries its sample, group, source rank, policy version and scorer version, and the child must echo them. An idempotency key derived from those fields and a fingerprint of the payload lets a retried request return the cached result instead of scoring twice. The child serializes scoring with its lifecycle operations and refuses non-finite outputs. The parent can keep the scorer resident or bring it onto the device only for the duration of each call. A child may host an entire vLLM engine (Kwon et al., 2023): for the EditScore judge, the merged integration (UniRL #348) reports that parking takes 5.1 s and leaves 1.0 GB of 47 GB on the device, that waking takes 0.3 s, and that scores are bit-identical across the cycle. Managed children currently accept image and image-editing requests only.

In a standalone service. A separate reward service places a gateway in front of Ray worker groups (Moritz et al., 2018). Each configured reward is a group of replica actors that reserve their own devices and install their own package set on first start. The gateway decodes a batch once, buckets its items by requested reward, issues one actor call per reward with round-robin replica choice, and awaits each reward under its own timeout. A reward that fails is reported per item and per reward next to the scores that succeeded, so one scorer cannot turn a whole batch into an error; the client treats any missing score as a failure of that row. The per-scorer environments of the gateway share the cluster's interpreter and PyTorch build, which is the limit that the managed child removes. The single-scorer server used for managed children can also be started by hand on another node, which is how the judges of Section 5.4 were served.

5.3 Overlapping reward with generation

In the synchronous diffusion loop the driver first has every data-parallel rank generate its whole shard and only then dispatches scoring. With a remote judge this wastes time twice: the training node idles while the judge works, and all ranks' requests reach the judge at the same instant and queue behind one another.

An optional worker-side role, `RewardStack`, changes the unit of work instead of the contract. It is built on the same worker as the rollout engine and the reward role, slices the frontier of its shard into micro-batches of a configured number of rows, and for each micro-batch calls the engine and then the reward in process. The scored parts are concatenated in the original row order, so group membership and credit assignment are unchanged, and the driver skips its own reward phase. A micro-batch may be a single row; if the engine packs a whole sibling group into one request, the micro-batch is extended to the end of the group it would otherwise split. (Section 6.3 removes that floor for the engine measured below.) With `overlap` enabled, scoring calls are queued in order on one background thread and generation never waits for a score; a scorer failure surfaces after the next micro-batch is generated.

The role fails closed on layouts it cannot serve. It needs the engine and the reward as siblings on one worker, so a separate rollout slab and a reward slab are rejected at startup, as are a reward that is not resident during generation and sequence- or tensor-parallel groups, in which several ranks would score one shard. Overlap carries one semantic condition: a scorer that draws from the process-wide random generator would interleave with the sampler's noise draws and make the rollout itself irreproducible, so such scorers must run in the serial schedule. The asynchronous trainers overlap by other means: the autoregressive one admits the next generation before it scores the collected batch, and releasing a diffusion rollout lane while a remote reward is pending is proposed in an open pull request.

5.4 Measured effect of the schedule

Setup. One node with eight H20 GPUs trains BAGEL (Deng et al., 2025) for image editing with LoRA, generating through eight data-parallel vLLM-Omni engines. A rollout has 32 prompts with eight edits each, or 32 rows per rank, which a micro-batch size of eight splits into four micro-batches. An EditScore judge (Luo et al., 2025) on a second node scores every source–edit pair over HTTP, in an 8B and a 72B variant, both as one tensor-parallel vLLM engine behind the single-scorer server. Runs last two or three rollouts; the first is discarded as warm-up and the rest are pooled over repeated sessions on two allocations of the same hardware. Times are the driver's phase timers. The image-editing wall-clock figures are estimates for the injected source conditioning of Section 6.1. Generation is scaled by the ratio between the two conditioning paths that Table 7 measures at sixteen rows per rank, assuming the ratio holds at 32 rows per rank and for every micro-batch. The reward-side times—the judge's service and queueing and the reward time that remains outside generation—are kept as measured, since the judge scores the same number of 512×512 pairs in the same request sizes. Where the judge has headroom the whole generation saving reaches the wall-clock; where it is saturated, only the earlier arrival of the first micro-batch does. The bundle states the model and keeps the estimate next to its inputs. The number of rollouts is small and the workload is single, so the table supports the direction and rough size of the effect for this shape, not a distribution. Thirteen of the runs also logged, inside each worker, when every micro-batch was generated and when every reward call was sent and answered, and the judge's server logged the arrival and completion of every request. The judge's service times, total work, waits and per-pair costs below come from these traces; arrival times and windows that depend on generation follow from the estimate and are labelled as estimated. **TODO[data]:** The image-editing rows of Table 3 and Figure 3 are estimates. Re-measure them with the injected conditioning (UniRL #547) at this shape: (i) both judges under the whole-shard, serial and overlapped schedules, at least three steady-state rollouts per arm; (ii) per-rank and judge-side traces of one run per arm, to refresh the judge-side numbers and restore the recorded timeline figure; (iii) the 16-row shape for both judges with whole-shard, eight-row requests without the stack, and two micro-batches per rank; (iv) optionally, the 8B-judge train phase with and without the collection pass. Tracked in UniRLReport issue #19.

Table 3: Generation plus reward wall-clock per rollout under three schedules: mean and range over steady-state rollouts. Editing rows use BAGEL with a remote EditScore judge; the last block is BAGEL text-to-image with an in-process PickScore. Overlapped rows pool the merged implementation, which queues every score, with an earlier prototype that kept at most one score in flight; the bundle reports them separately. †Estimated for the injected conditioning of Section 6.1 from the measured reward-side times; the rollout count is that of the underlying runs.

Schedule	Rollouts	Generate + reward (s)	Range (s)	vs. whole shard
EditScore 8B, remote node				
Whole shard, driver-dispatched reward†	4	102.0	101.1–103.0	–
Micro-batches, serial†	5	86.9	86.3–87.3	−14.8%
Micro-batches, overlapped†	7	82.3	82.0–82.7	−19.4%
EditScore 72B, remote node				
Whole shard, driver-dispatched reward†	2	155.0	154.5–155.4	–
Micro-batches, serial†	4	138.8	138.2–139.8	−10.4%
Micro-batches, overlapped†	6	140.1	138.1–142.9	−9.6%
PickScore, in-process				
Whole shard, driver-dispatched reward	2	56.0	56.0–56.0	–
Micro-batches, serial	2	52.4	52.4–52.5	−6.3%
Micro-batches, overlapped	2	52.3	52.3–52.3	−6.5%

A judge with headroom. With the 8B judge, serial micro-batches reduce generation plus reward from 102.0 s to 86.9 s (14.8%), and overlapping reduces it to 82.3 s (19.4%). Generation itself takes about 73 s in every arm (Figure 3); the schedules differ in where the reward time falls. After a whole shard, the eight requests arrive together and the server scores them one at a time, 3.08 s each, so the last rank waits 24.6 s for a score that took the judge three. With serial micro-batches the ranks collide only once, on their first micro-batch, where the longest wait is 9.6 s. The unequal waits leave the ranks staggered, and each later request finds the judge free and returns in about 1.3 s. With overlap, generation never waits, so the ranks stay in step and collide on every micro-batch; those waits now run beside the next micro-batch's generation, and only the last one remains after generation ends. The judge's 37.7 s of work per rollout fits in the estimated 54.8 s between the first micro-batch's arrival and the end of generation, and that headroom is what overlap uses.

A saturated judge. With the 72B judge, serial micro-batches reduce generation plus reward from 155.0 s to 138.8 s (10.4%), and the overlapped schedule is indistinguishable from the serial one: 140.1 s, with a range of 138.1–142.9 s against 138.2–139.8 s. In both, the judge works without a gap from the arrival of the first micro-batch, an estimated 18.2 s into the rollout, until its last request. The rollout therefore ends at that arrival plus the judge's total work, whatever order the requests come in. That work was 119.7–121.4 s per traced rollout under the serial schedule and 119.5–124.5 s under the overlapped one, and it drifts by up to three seconds between consecutive rollouts of one arm, so the few seconds between the two rows of Table 3 are not an effect of the schedule. Once the judge is busy for the whole rollout, adding judge capacity or making its requests cheaper is the remaining lever.

How one gets into that regime. The regime is defined by the ratio of reward cost to generation cost, not by the judge's parameter count, and this deployment reaches it in a way a production one need not. A 72B judge scoring for a policy with 7B active parameters is an unusual pairing, and one judge replica serves all eight training ranks. Even at the per-pair cost of whole-shard requests that replica would need 76.9 s per rollout, more than the estimated 54.7 s between the first micro-batch's arrival and the end of generation, so it is saturated at this shape whatever the request size. The single-scorer server deepens this: it scores one request at a time, so the small requests that arrive concurrently from different ranks are never merged into one forward. On the same judge a 32-pair request costs 0.30 s per pair and an eight-pair request 0.47 s, so serving a rollout as 32 small requests raises the judge's total work from 76.9 s to 120.4 s. Giving this judge headroom at this shape takes more than one replica; batching across concurrent requests or splitting prefill

from decode reduces how many. We report the arm as a stress test of the schedule, not as a shape to plan for.

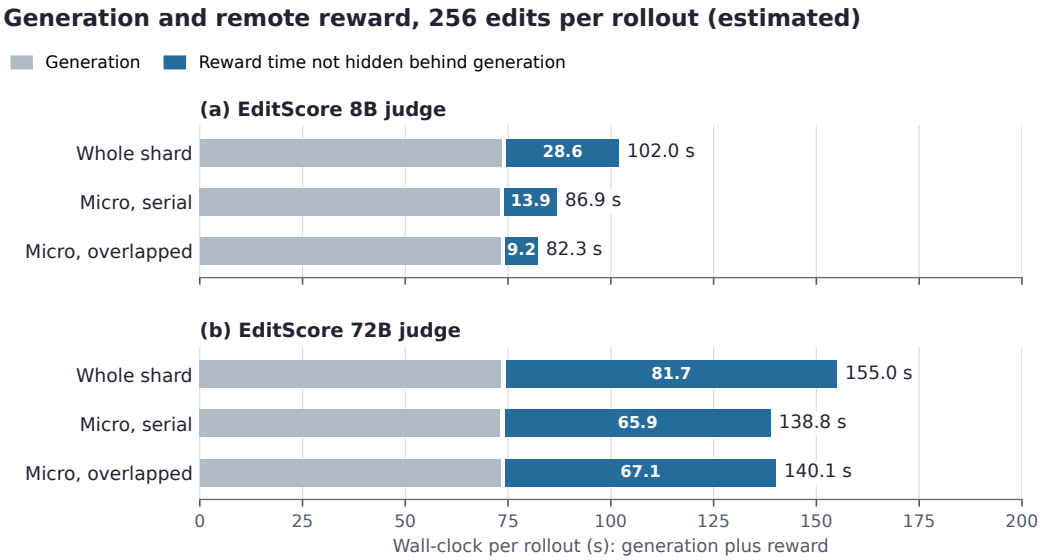


Figure 3: Where the rollout's wall-clock goes for the two remote judges of Table 3, estimated as in the table. Generation is unchanged by the schedule; the schedules differ in how much reward time remains outside it.

The price of small requests. Micro-batching increases the judge's work. An eight-pair request costs the 72B judge 3.76 s and a 32-pair request 9.61 s, so a rollout scored in micro-batches needs 120.4 s of judge time against 76.9 s for whole shards; for the 8B judge the figures are 38.5 s and 24.6 s. The schedule wins because that work starts a quarter of the way into generation instead of after it, not because there is less of it. Smaller requests without the stack only pay the price: in a shallower shape with 16 rows per rank, splitting the whole-shard request into eight-row requests was slower than the baseline on both judges (52.2 s to 57.2 s and 86.1 s to 100.7 s, estimated as above). Per pair the same judge is 0.47 s against 0.30 s, and the 8B judge 0.150 s against 0.096 s: the penalty is the serving path, not the schedule. The single-scorer server used here scores one request at a time, so requests that arrive together from different ranks wait rather than batch. Continuous batching across them, a separate prefill stage, or simply more replicas would remove this cost and leave the schedule's arrival stagger as a pure gain.

An in-process scorer. With PickScore in the policy's workers, reward is a small part of the rollout, and the stack reduces generation plus reward from 56.0 s to 52.4 s. Scoring inside the worker took about one second in these runs; the rest of the baseline's reward phase is the driver-dispatched call. Overlap has nothing left to hide here.

A side effect on training. The micro-batch loop leaves garbage in the worker's Python heap. When the following train phase started without a collection pass, it took 192.2 s in the 8B-judge runs and 58.0 s in the PickScore runs; with a pass at the rollout boundary it took 178.1 s and 54.5 s. The merged role therefore collects once the shard has left the worker. In both workloads the train phase replays against the trainer's own conditioning, so these times are measured rather than estimated. The cost of a rollout schedule has to be read off the whole step, not the rollout phase alone.

5.5 Sharing one GPU among small scorers

The standalone service gives every scorer actor whole devices. That is simple and isolates faults, but a panel of small preference models leaves most of a large accelerator idle. Ray can place several actors on one device through fractional resources; the fraction is scheduling bookkeeping, and the separate CUDA processes it

co-locates are time-sliced. NVIDIA's Multi-Process Service (MPS) lets kernels from different processes run concurrently (NVIDIA Corporation), which keeps one process, and one dependency set, per scorer while using the idle capacity between their kernels.

The open pull request adds this as an opt-in runtime. The service either owns the MPS daemon of its node, starting it before any actor creates a CUDA context and stopping it after the last one has exited, or attaches to a daemon owned by the platform. Each reward may set an active-thread percentage and a device-memory allocation cap, which are exported to its actor before its first CUDA call. The configuration is rejected when it combines sharing with tensor-parallel scorers, with scorers that have not been qualified, or with half-precision weights below a full active-thread share. Shutdown stops admission, waits for submitted GPU work, and only then terminates actors and the daemon. The cap limits what a client may allocate; it is not memory isolation, and all clients of one device share a fault domain, which the health endpoint reports.

Table 4: Throughput gain (%) of MPS over fractional sharing without MPS for two or three copies of one scorer on a single H20, as reported in pull request #492; b is the model batch. Dashes mark batch sizes at which the scorer does not run one batched forward of that size.

	Two copies			Three copies		
Scorer	b=1	b=4	b=8	b=1	b=4	b=8
CLIP	+11.2	+20.0	+14.8	+16.5	+24.4	+19.2
HPSv2	+17.4	–	–	+48.4	–	–
ImageReward	+24.0	–	–	+40.8	–	–
HPSv3	+20.6	+11.2	–	+22.6	+12.8	–
GOT-OCR	−2.8	–	–	−3.2	–	–

Reported results. Table 4 transcribes the author's qualification on H20 GPUs; we did not repeat it. For panels of different scorers on one device, the pull request reports a median gain of 24.9% for three light scorers and 9.6% for two, smaller gains when a heavier vision–language scorer bounds the panel, and none for an autoregressive OCR scorer. Scores were identical with and without MPS on its deterministic fixture. The gain comes from filling idle device time between the kernels of different processes; the pull request's profiles put a single CLIP process at about half of the device's time at batch one. It is a utilization result for small models and does not add capacity once one scorer saturates the device.

A correctness boundary. The same qualification found that an active-thread percentage below 100 produced wrong half-precision matrix products on the tested H20 driver stack: non-finite CLIP scores, and finite but incorrect PickScore values. The runtime therefore refuses half-precision scorers below a full share. A reviewer did not reproduce the failure on a different accelerator and driver, where the same panel was bound by CPU and HTTP work and co-location cost almost nothing even without MPS. The benefit and the hazard both depend on the device and driver, and a throughput figure from one stack should not be carried to another. Only CLIP and PickScore at a full share are qualified in the pull request; vision–language judges, longer soak runs, and sharing a device between reward and the policy's own roles are outside its scope.

6 Rollout Fidelity and Request Shaping

Section 5 took the generated sample as given and asked where its reward should be computed. This section looks at the sample itself. Two properties of a rollout decide whether the reward and the gradient refer to the same thing, and what the rollout costs: the engine and the trainer must condition on the same sequence, and the engine's request must be shaped so that one request is not one sample when a group of siblings would do. Both are visible only through the engine's own boundary, which is where a lineage-preserving trajectory does not help: the `Sample` that comes back is well formed either way.

Source scope. Four changes to the vLLM-Omni BAGEL path, proposed as UniRL pull requests #547–#550 and open at the time of writing. They are stacked in that order and must land in it. Every

measurement below is ours, on one eight-GPU H20 node, and the per-run records and generator are under `artifacts/reported_results/2026-10-04-rollout-fidelity/`. The scheduling experiment of Section 6.5 is on a branch with no pull request.

6.1 A conditioning mismatch that the importance ratio could not see

In image editing the policy conditions on a source image. UniRL builds that conditioning itself—its own image transforms, its own encoder—and injects the resulting key–value contexts into the serving engine, because the trainer rebuilds the same contexts when it replays the sample. Since the engine was migrated to a newer serving version, three helpers in the BAGEL pipeline kept reading a field of the request *batch* after the caller had already unwrapped it to a single request. The field was therefore always absent, the source image was never found, the injection never ran, and the engine fell back to its own image-to-image path: a different image transform at a fixed resolution, a sampled rather than an averaged latent, and the source's dimensions as the output canvas.

Nothing in the training loop noticed: every image-editing rollout on this path was generated under one conditioning and replayed under another, with a 5938-token prefix on the engine against 2263 on the trainer at the same source size. Why it went unnoticed bears on what a parity check has to measure. The natural check is the importance ratio: before the first optimizer step of a rollout, the policy has not moved, so the ratio against its own behavior anchor should be 1 up to numerical noise. It was. With the mismatch the largest first-update deviation was 5.0×10^{-5}; without it, 3.4×10^{-5}; a healthy image-editing run reached 5.4×10^{-5} (Table 5), so the mismatched run lay inside the healthy range. The cause is how the log-probability of a denoising transition is reduced. Both the engine and the trainer average the Gaussian log-density of a transition over every latent element instead of summing it, so a difference in the predicted mean, even one large enough to change what is generated, reaches the ratio divided by the number of latent elements, which is in the tens of thousands at 512×512. Under this reduction the ratio cannot separate a conditioning mismatch from numerical noise. A summed log-probability would not dilute the difference in this way; we have not measured whether the mismatch would then be visible. Under the recipes' default anchor, which recomputes the old log-probabilities from the trainer itself, the ratio is identically 1 and could not have seen anything at all.

What does see it is a structural check. The engine already hands the driver a hook where the driver's initial noise is substituted; the same hook now records three integers and the canvas—the length of the generation context, its positional offset, and the rendered size—which ride the existing capture channel into the trajectory's model conditions and are compared, per row, against the context the trainer rebuilds at replay. A mismatch raises at the first replay, with both sides printed. It costs nothing, because replay builds that context anyway, and conditions that carry their own contexts are not checked. Table 5 lists what it catches. The ratio guard is kept as a second, coarser check, opt-in through a tolerance and warning rather than raising: it is meant for a policy that is not the one that generated the rollout—unsynchronized weights, the wrong adapter—and, with the averaged log-probability, does not detect a conditioning mismatch.

Table 5: Probes of the token-layout check. Lengths are tokens of the generation context on the engine and on the trainer. The last column is the largest first-update deviation of the importance ratio from 1 in the same run, which is the check this replaces.

Probe	Path	Engine	Trainer	Layout check	$\lvert r-1\rvert$
Main before the injection fix	it2i	5938	2263	raises at the first replay	5.0×10^{-5}
Trainer prompt lengthened by three tokens	t2i	63	66	raises at the first replay	–
Trainer source image halved in width	it2i	2268	1161	raises at the first replay	–
After the injection fix	it2i	2263	2263	passes	3.4×10^{-5}
Healthy run (rollout anchor)	t2i	equal	equal	passes	8.6×10^{-6}
Healthy run (rollout anchor)	it2i	equal	equal	passes	5.4×10^{-5}
Healthy run (replay anchor)	it2i	equal	equal	passes	0

Restoring the injection changed the rollout twice over. Generation of sixteen edits on one rank fell from 49.2 s to 36.8 s, because the engine's own path encodes the source into four times as many tokens as UniRL's. And the judge's verdict on the result moved: the mean EditScore of the first rollout, before any optimizer step and on the same prompts, went from 2.80 to 4.62 (Figure 4). That second number is not a learning result—no update had happened—but a measure of what the engine produces under each conditioning.

6.2 A canvas per group

A rollout batch has had one output resolution, fixed by the recipe, because the trajectory's latent field is one dense tensor and the engine is given one size. Production prompt sets are not like that: a dataset of real prompts carries its own natural sizes, and training at one of them either wastes compute on prompts that do not need it or refuses the ones that do.

A dataset row may now carry its own canvas in its metadata. The trajectory resolves the canvas per frontier row from its root, falling back to the recipe's size, and the engine's adapter sends one call per contiguous run of same-canvas rows. The latent field stays dense: a smaller canvas's trajectory is right-padded to the default canvas's token count and the replay trims it back, so the recipe's size is an upper bound and a larger canvas is rejected. A prompt group keeps one canvas, which is what leaves credit assignment, the reward micro-batches of Section 5.3 and the scorers untouched. No configuration key is added, and a dataset without the field produces the calls it produced before. Engines that render one size per request—the train-side one, the other serving adapters, the composed and video paths—reject the field by name instead of ignoring it, which matters because ignoring it would train on images of a size nobody asked for.

The mechanism is bounded in a way worth recording. It supports the algorithm family whose replay anchor is a scalar per transition. Objectives that store an anchor per token, and those that read the latent field directly, still need one canvas per rollout; they now fail with a named error where they used to fail on a tensor size.

Table 6: Cost of one group of four siblings at three canvases, seconds, on one rank. Image editing additionally encodes a 512×512 source. The 768×768 text-to-image cell was measured on an earlier revision of the same branch.

Canvas	Image tokens	it2i, one per sample	it2i, packed	t2i, packed
512×512	1024	9.05	7.25	6.50
768×768	2304	17.45	15.76	14.55
1024×1024	4096	30.48	28.83	27.00

Table 6 is the reason mixed resolution is a scheduling problem and not only a capability. Cost is close to linear in image tokens, and the span between the cheapest and the most expensive canvas in one batch is a factor of four. With two groups per rank drawn from a mixed dataset, the slowest rank of a rollout took 1.16 to 1.84 times the mean across ranks, and every rank waits for the slowest.

6.3 One request per group of siblings

Group-relative credit needs several samples per prompt, and for image editing each of those samples was its own engine request. Every request rebuilt the same three contexts from the same source image, encoder passes included. Where the guidance scales allow it, the adapter now sends one request per contiguous run of a group's rows, and the engine builds the contexts once and replicates the generation context per sibling. Generation of sixteen edits per rank fell from 36.5 s with the per-group canvas alone to 28.4 s, about 22% (Table 7), for roughly two gibibytes more peak memory per GPU; one group of eight went from 18.04 s to 13.96 s. Text-to-image already packed and is unchanged.

One consequence reaches back into Section 5.3. The initial noise is addressed per row, so a packed request may cover any contiguous part of a group rather than a whole one. The engine therefore no longer declares that it packs groups, and a reward micro-batch is free to cut one: a four-row micro-batch of an eight-sibling

group is a single four-output request. The floor that Section 5.3 describes for this engine is removed by this change.

6.4 What the four changes do together

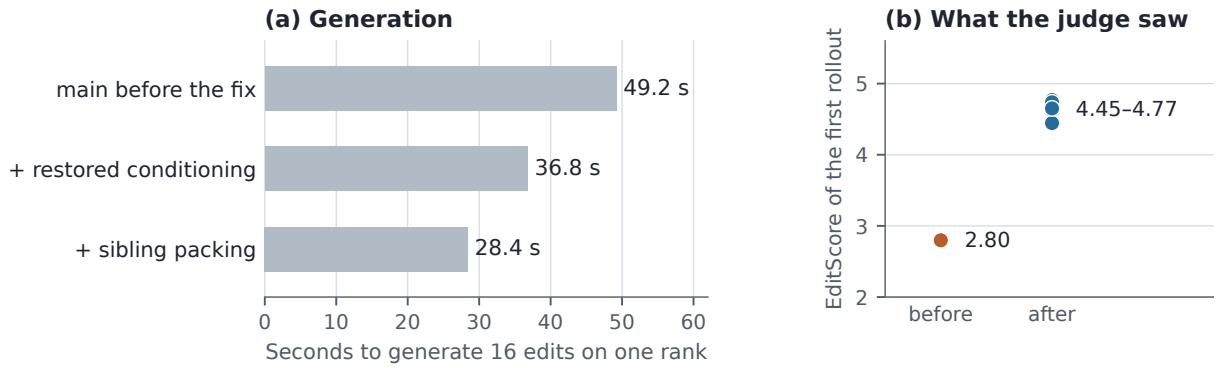


Figure 4: The image-editing rollout before and after the changes, on one eight-GPU node with a remote EditScore judge. Left: seconds to generate one rank's sixteen edits. Right: the mean judge score of the first rollout, which no optimizer step has yet influenced; the points after the fix are separate runs of the same first batch.

Table 7: Image editing with LoRA on one eight-GPU H20 node, BAGEL through vLLM-Omni, sixteen prompts with eight samples each at 512×512, EditScore 8B on a second node. Generation is the slowest rank's seconds for its sixteen rows. The reward column is the mean judge score of the first rollout on the same prompts, before any optimizer step.

Rollout path	Rollouts	Generation (s)	vs. `main`	First-rollout reward
`main` before the fix	1	49.2	–	2.80
+ restored it2i conditioning (#547)	1	36.8	−25%	4.62
+ per-group canvas (#549)	1	36.5	−26%	4.77
+ sibling packing (#550)	4	28.4	−42%	4.45–4.74

Table 7 puts the four changes on one workload: generation per rank falls 42%, from 49.2 s to 28.4 s, and the first rollout's judge score settles between 4.45 and 4.77 instead of 2.80. The first-update ratio stays within 10^{-4} of 1 throughout. The layout check is always on from #548 and raises at the first mismatched replay; every arm from the per-group canvas on completed its replays, including the one where a micro-batch cuts a group. These are one to four rollouts per arm on a single workload shape: they establish the direction and size of each change on this path, not a distribution.

6.5 The long tail that mixed resolution exposes

Once a rollout can mix canvases, the assignment of rows to ranks stops being neutral. Today it is the data-parallel split: prompts in order, equal row counts, no attention to cost. On a dataset whose canvases are drawn per row from a production-like mix with a four-fold span in tokens, the steady text-to-image rollout spent 29% of its rollout window with at least one rank idle, waiting on a rank that had drawn the expensive prompts.

We measured what a cost-aware assignment recovers. A coordinator hands each rank units of work from a queue ordered by an estimated cost, and an idle rank may take the cheapest unit from the busiest rank's

queue; rows are restored to their original order by identifier before credit assignment, so the trajectory the trainer sees is unchanged. Table 8 reports the steady rollout of each arm.

Table 8: Assignment of mixed-canvas work to ranks, second rollout of each arm, on one eight-GPU node. "Idle" is the fraction of the rollout window in which at least one rank has finished and waits. The cost estimate is image tokens. This mechanism is on a branch with no pull request.

Assignment	Rollout + reward (s)	Slowest rank (s)	Mean rank (s)	Idle
t2i, in-process PickScore				
Prompt-order shards (today)	150.2	147.2	103.8	29%
Cost-balanced shards	108.6	105.8	103.7	2%
Cost-balanced shards with stealing	108.7	105.8	103.6	2%
it2i, EditScore 8B on another node				
Prompt-order shards (today)	99.3	91.0	63.7	29%
Prompt-order shards, reward overlapped	93.7	91.2	63.7	30%
Cost-balanced shards with stealing	77.4	68.0	63.7	6%
Cost-balanced with stealing, reward overlapped	71.5	68.1	63.8	5%
it2i, EditScore 72B on another node				
Prompt-order shards (today)	128.4	90.8	63.7	21%
Cost-balanced with stealing, reward overlapped	131.1	68.4	63.8	8%

Three readings, each from two rollouts of each arm. Balancing recovers most of the tail where the cost estimate is good: text-to-image falls from 150.2 s to 108.6 s and idle time from 29% to 2%, while the mean per-rank generation time is unchanged at about 104 s, so the work was redistributed rather than reduced. Stealing is insurance against a wrong estimate rather than a gain in its own right: with a good cost model it never triggered in text-to-image and moved at most 2 units per rollout in image editing, and with a deliberately naive one that counts rows and ignores the canvas, which gave back 25 s of the balancing gain, it recovered 18 of them (133.9 s to 115.9 s). And the two mechanisms compose with the reward schedule of Section 5: against a judge with headroom, overlapping reward alone took the editing rollout from 99.3 s to 93.7 s, the cost-aware assignment alone to 77.4 s, and both together to 71.5 s, or 28%.

Against the saturated 72B judge, cost-aware assignment made rollout plus reward *longer*, with or without overlapping reward: from 128.4 s to 131.1 s with overlap and 132.2 s without in the steady rollout, and from 118.3 s to 131.5 s and 133.3 s in the first. It still flattened generation—the slowest rank fell from 90.8 s to 68.4 s—but that is exactly the harm: with ranks finishing together they reach the judge together, and the queueing that the uneven baseline had spread out now happens all at once. This is the same mechanism that Section 5.4 found behind micro-batching's gain, read in reverse. Two consequences follow. Such a scheduler has to be gated on measured reward headroom rather than enabled by default. But the regime that makes it harmful is the one Section 5.4 shows this deployment manufactures: one judge replica, and a server that scores one request at a time, which turns the scheduler's small concurrent requests into a queue instead of a batch. With enough judge capacity the gate would rarely close; Section 5.4 finds that at its shape one replica of this judge is saturated even with whole-shard requests, so the capacity has to come from more replicas, with batching across concurrent requests or a disaggregated prefill stage reducing how many. The arm belongs in this report as the boundary of the mechanism, not as a configuration to expect. A second cost, measured and not yet addressed: in every arm that reassigned rows, the training phase ran 1–5% slower in the steady rollout and 3–12% slower in the first. Part of this is the queue path itself: in the text-to-image runs, prompt-order shards routed through the same queue, which reassigns nothing, trained 2–3% slower. The rest is consistent with a rank replaying rows that another rank generated, whose latents cross workers once the rows are restored to their original order; we have not separated the two. In the steady rollout the whole step is still shorter wherever the assignment shortens rollout plus reward. In the first rollout the slower training phase can outweigh that gain: with the 8B judge, cost-aware assignment without overlap made the first step 2.7 s longer. An assignment that also followed the rows into training would avoid the replay cost. None of this is proposed yet.

7 Instantiations of the Common Contracts

The following paths show which information crosses the common boundary and which computations remain specialized. They illustrate the contracts at the design level, as implemented in the audited source, rather than inventory the system's capabilities. Section 8 exercises only the group-relative AR objectives, Flow-GRPO, and the frozen-rewrite cross-stage path; the GAE, DiffusionNFT, rewrite-training, and agentic paths described below have stock recipes but no reported training runs.

7.1 Autoregressive reasoning

An input part carries prompt text and metadata. Forking by group size creates generated shells with sibling IDs. An AR engine fills packed token segments and decoded text, and a reward role attaches scalar or component rewards. Group-normalized credit is computed on the driver before the batch is scattered; token-level generalized advantage estimates are instead computed per trajectory on the training workers, after the critic anchor is frozen. The train stack replays the same segment and applies the selected objective. Optional token-length balancing permutes individual generated rows of the training layer before the train stack scatters them, after driver-side group credit but before per-trajectory GAE, which does not depend on row order. It keeps each row's identifier and any attached advantage but abandons sibling contiguity, so no grouping operation may follow it; the E3-AR UniRL recipe enables it.

7.2 Diffusion and flow generation

A diffusion request forks prompts and records sampling parameters, selected SDE indices, and an initial-noise recipe; the hosting engine pins the sigma schedule onto the shell before generation. The engine returns decoded media plus a latent segment whose sparse trajectory fields are sufficient for the selected update. FlowGRPO-style objectives replay recorded steps and compare new transition log probabilities against a frozen rollout or replay anchor (Liu et al., 2025). Forward-process objectives such as DiffusionNFT use clean generated samples rather than the same reverse-trajectory likelihood path (Zheng et al., 2025). The common train stack handles microbatching and optimizer state; the algorithm remains responsible for these incompatible estimators.

7.3 Cross-stage and environment-interacting trajectories

In a prompt-enhancement path, one root produces one or more AR rewrites, and every rewrite produces one or more images (Equation 4). The image reward is attached to the leaf; when the rewrite stage trains, parent-major reduction supplies its feedback. The primary E4 case freezes that stage and uses ancestry to select the diffusion advantage groups. The prompt-enhancement workflow needs this ancestry to survive rollout scheduling and sharding; flattening it into unrelated rows would require rebuilding that correspondence out of band. In a tool-interacting path, each trajectory is a single-branch chain: the policy emits an action-like text part, the environment appends an observation part, and the process repeats until termination or suspension. The agentic trainer keeps one `Sample` per trajectory, forms a group from repeated copies of the same root rather than forked siblings, computes group advantages over those trajectories, and trains on their flattened generated turns, each inheriting its trajectory's advantage, so lineage there conditions each turn within a trajectory rather than defining groups or shards.

E4 checks these groups against an explicit ID-based reference on synthetic fixtures, leaving the recorded-rollout check open, and then studies the two grouping objectives in training.

8 Evaluation: Reported Results and Remaining Evidence

We organize evaluation around four research questions. Each reported run must archive the resolved configuration, commit and dirty diff, model and dataset revisions, environment manifest, raw logs, structured metrics, parser version, and evaluation outputs. Primary comparisons use the same hardware one framework at a time. The operational commands, run-directory schema, and failure-routing rules are frozen in `GPU_HANDOFF_README.md`; the stricter evidence-admission contract is in `EXPERIMENT_RUNBOOK.md`.

Evidence scope and date. The legacy online experiment report, retrieved on 2026-09-27, states an as-of date of 2026-09-22. We combine it with the result manuscript in pull request 2 at commit `ecce5097`; source snapshots, transcribed data, and a deterministic figure/table generator accompany this repository under `artifacts/reported_results/2026-09-22/`. The E2 and diffusion E3 tables below are regenerated from those *reported summaries*, not from newly inspected training logs, and the underlying per-run repositories were unavailable during that integration. The later E3-AR study was rerun and is archived separately under `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/`, including resolved recipes, the recipe and effective-work audit, run identities, post-warmup per-step records, phase and band summaries, and the deterministic plotting script. The HunyuanImage-3.0 and MiniMax-H3 recipe reproductions in Section 8.1 are likewise generated directly from their per-step W&B histories and resolved training recipes. We retain the distinction between report-derived and directly exported evidence, avoid reconstructed legacy learning curves or unsupported uncertainty bars, and list unresolved source discrepancies in `EXPERIMENT_RESULTS.md`. The settings below describe the prespecified protocol unless explicitly identified as an observed result; matching each setting to a resolved run configuration remains an artifact-release obligation.

Table 9: Experiment-to-claim map and evidence status through 2026-10-11. P0 rows remain necessary for the primary trajectory-centric claim.

ID	Priority	Controlled study	Paper evidence
E0	P0 gate	Correctness, configuration, and instrumentation checks	Correctness closure reported; evidence packaging remains incomplete
E1	P0	Qwen3-4B-Base on DAPO-Math	GRPO and DRPO each trained at one seed; primary external endpoint missing
E2	P0	SD3.5-Medium diffusion training	Three-seed UniRL scores and diversity reported; reference trained, not evaluated
E3	P0	Native UniRL–VeRL-Omni recipes, eight GPUs	Aggregate timings reported; strict semantic parity not established
E3-AR	P0	Recipe-matched Qwen3-4B GRPO, UniRL–veRL, 16 GPUs per system	500 steps complete; resolved recipes, parity audit, per-step measurements, and curves released
E4	P0	Root versus rewrite grouping	Synthetic equality checks pass; no consistent three-seed quality advantage
E5	P0 minimum	Representation cost and deployment topology	Topology measured; paired representation cost not measured
E6	P1	Synchronous/asynchronous Qwen3 study	Single-seed pilots only; formal sweep canceled

The order is E0, then E1–E4 and the minimum E5 topology/cost slice; E6 and broader E5 scaling/transport backends follow. If compute is limited, we complete primary E0–E5 before adding more models, algorithms, or scaling points: without E4 the paper would not directly test lineage, and without the minimum E5 slice it would not bound the cost of that abstraction.

Environment and source identity. E3 and E5-T report a fixed total allocation of eight GPUs; the complete hardware, topology, environment, source-patch, model, data, and scorer manifests still need to accompany those legacy results. E3-AR instead uses two eight-GPU H20 nodes per framework and releases its pinned source identities and resolved recipes with the per-step records. The older E1 incident report describes a particular 32-GPU allocation, but its hardware must not be silently reused to fill the E2/E3 manifests. Likewise, the design-audit commit is not a substitute for the later experiment commits.

8.1 RQ1: Does UniRL learn on representative workloads?

The first question is scientific rather than purely operational. A complete training-validity claim requires improvement over the frozen base together with satisfactory collapse diagnostics. E2 reports an improvement in automatic scores but triggers the diversity guard, so its result is a qualified quality–diversity trade-off. Reproducing another implementation additionally requires a full-budget, mathematically aligned reference with common quality endpoints; a systems speed run alone cannot supply that evidence. The E2-R reference is trained but has not been evaluated head-to-head, while the adapted AR systems launcher now exists but still lacks the aligned external quality endpoint.

AR workload. E1 trains Qwen3-4B-Base (Yang et al., 2025) with full-parameter GRPO on DAPO-Math-17k (Yu et al., 2025). The data name does not imply use of the DAPO optimizer; we fix the population-standard-deviation group estimator implemented in the audited recipe. Each rollout contains 64 prompts with eight samples (512 trajectories), uses SGLang TP=1 with thinking enabled, temperature/top-p 1.0, top-k disabled, and at most 8192 new tokens. The learner uses group-standard-deviation normalization, symmetric clipping at 0.2, `seq-mean-token-mean` aggregation, four updates per rollout, a 10,240-token microplanner, AdamW learning rate 10^{-6} and weight decay 0.01, and full-weight publication every rollout. We prespecify 800 rollouts on 32 total GPUs, internal AIME evaluation every ten rollouts, full checkpoints every 200 rollouts, and seed IDs 11, 22, and 33 applied to data order and the SGLang server RNG. These seeds and the checkpoint cadence must be supplied as run-time overrides because they are not stock-recipe values (the stock data-source seed is 42); at the audited release a per-request SGLang sampling seed requires deterministic inference, so the server RNG must be seeded through engine arguments recorded in the resolved configuration. The planned external evaluation covers the frozen base and checkpoints 200/400/600/800 on MATH-500 (500 problems, avg@4) and AIME 2024/2025 (30 problems each, avg@16), using the registry's fixed decoding and `math_verify` grader. The UniRL run and the selected current reference must align prompts, chat template, verifier, maximum generation length, group size, effective batch, clipping, KL treatment, optimizer updates, and evaluation decoding. The public veRL Qwen3 FSDP example is only a scaffold because its stock model variant, data, group size, response budget, and KL/objective settings differ. E3-AR uses a pinned adapted launcher whose resolved recipes are archived and checked field by field against fifteen cross-framework effective-work identities in `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/recipe_parity_audit.json`; two residual estimator-level differences that configuration cannot remove are quantified there, and the bundle README lists the recipe items these identities do not cover. Absent frozen-base and checkpoint evaluations under the common external protocol, it does not by itself establish quality reproduction. The final 800-rollout checkpoint is primary. Internal evaluation uses AIME 2024/2025, so an AIME-selected best checkpoint is reported only as validation-selected and cannot establish an untouched AIME test result. MATH-500 is the primary external AR endpoint, with a training/evaluation overlap audit required before interpretation; the fixed-budget AIME curves are secondary diagnostics. The protocol calls for every seed, final-checkpoint curves, and 95% confidence intervals; avg@k denotes mean accuracy across k sampled answers, not pass@k.

E1 status: training complete, primary endpoint open. The current report records two full-budget single-seed arms: GRPO and an additional DRPO arm. Internal AIME changes from 0.0208 to 0.0177 for GRPO and from 0.0250 to 0.159 for DRPO. Both improve and then decline during training; these are validation observations under a different decoding budget from the primary MATH-500 endpoint. The external endpoint and across-seed replication remain missing, so the observations do not establish objective superiority or general AR training validity. The additional DRPO recipe and selection history also need to be documented alongside the original GRPO protocol. Issue 1 records a historical evaluation hang; the later report attributes its resolution to the bounded grader-process cleanup in upstream pull request 462 and records completed training. The older issue is therefore not evidence that E1 is still blocked before rollout zero.

Diffusion workload. E2 fixes SD3.5-Medium (Stability AI, 2024) and the UniRL FlowGRPO setting used by E3, i.e. the stock `sd3_vllmomni.yaml` recipe with the overrides in `run_unirl_sd35_aligned.sh`: 48 prompts with 16 images (768 samples) per rollout, 384×384 training resolution, ten denoising steps, three SDE steps in the first half, $\eta = 0.8$, guidance 1, and distinct initial noise. PickScore (Kirstain et al., 2023) supplies the training reward; advantages subtract each prompt-group mean and divide by the batch-wide reward standard deviation. LoRA rank/alpha is 32/64 on eight matched attention projections; the learner uses two updates, microbatch eight, learning rate and weight decay 10^{-4}, clip 10^{-5}, and publication every rollout. We prespecify 300 rollouts on one eight-GPU node with seed IDs 11, 22, and 33. The same budget and seed IDs apply to the pinned VeRL-Omni reference; checkpoint export and estimator parity are audited before admission. The stock pair differs in SDE strategy, step selection, and potentially normalization/anchor semantics, so it currently defines a related-recipe comparison. At least one full-budget E1 or E2 reference must pass mathematical and work parity before the paper claims reference-implementation reproduction; otherwise that claim remains unestablished. Frozen base and every 50-rollout checkpoint use a

shared diffusers evaluation pipeline. The primary image endpoint is PartiPrompts HPSv3 (Ma et al., 2025b); ImageReward (Xu et al., 2023) and compositional scores provide complementary checks. These are held-fixed automatic judges distinct from the optimized PickScore model, not independent human studies.

Image evaluation identity. The registry key `image/geneval2` points to UniRL's `synthetic/test.jsonl` and a Qwen3-VL-8B Soft-TIFA-style scorer, served by the reward service by default or run in-process with `-local-geneval2`. We name this the *synthetic compositional set*; it is not an official GenEval 2 result (Kamath et al., 2025) without a separate data-and-scorer parity audit. The prespecified image endpoints use 512×512, 40 steps, guidance 1, and an evaluation seed of 42 shared across training seeds and checkpoints. The later E2 report describes 16 images for each of 1,632 preference prompts, rather than the original one-image preference endpoint; the exact sampler and aggregation revision must be reconciled in the released protocol. The synthetic endpoint additionally fixes its explicit linear sigma grid, prompt-hashed seeds, and 256-token text-encoder limit; the preference endpoint fixes the pinned pipeline schedule and index-based seeds. Dataset sizes, revisions, overlap counts, and scorer hashes remain open manifest fields.

The diversity guard is within-prompt mean pairwise LPIPS (Zhang et al., 2018) (AlexNet features), computed over 16 images per PartiPrompts prompt at the same evaluation setting, with identical image seeds for base and checkpoint. We average all 120 pair distances per prompt, then report the relative change in the mean across prompts and a paired prompt-bootstrap interval. A greater than 10% relative decrease is a study-specific investigation trigger; it is not a literature-established quality threshold. A near-zero base makes this relative criterion undefined and requires the absolute change to be reported. This guard diagnoses reduced diversity and is not itself a perceptual-quality metric.

Table 10: Reported E2 diffusion endpoints after 300 rollouts, with three training seeds. Changes are absolute score differences unless marked as relative percentages. Missing base/final levels are not reconstructed from differences.

Metric	Base	Rollout 300	Δ score	Δ relative
HPSv3	1.2024	9.8126	+8.6102	–
ImageReward	–	–	+1.3932	–
PickScore	–	–	+0.0970	–
Composition (VQAScore)	0.1011	0.1552	–	+53.5%
LPIPS diversity	–	–	–	-24.79%

E2 result: higher automatic scores with lower diversity. The reported mean HPSv3 gain is 8.6102, while within-prompt LPIPS changes by -24.79%. Figure 5 shows the available individual-seed differences, keeping score improvement and diversity reduction adjacent. The report records complete scoring with no errors; the preference evaluation contains 1,632 prompts and 16 images per prompt, not that many independent training runs. Synthetic composition and preference use different evaluation sets, so the preference image count must not be assigned to every metric. All three seeds exceed the prespecified diversity-investigation threshold. The reported reduction is already present at rollout 50; stopping at that checkpoint does not eliminate it, although these observations do not rule out earlier stopping or a different intervention. External automatic judges also improve, but this does not establish human preference or exclude reward exploitation. The supported conclusion is a score–diversity trade-off, not an unqualified training-validity or no-collapse result. Only endpoint summaries are available in this bundle; no intermediate learning curve is interpolated.

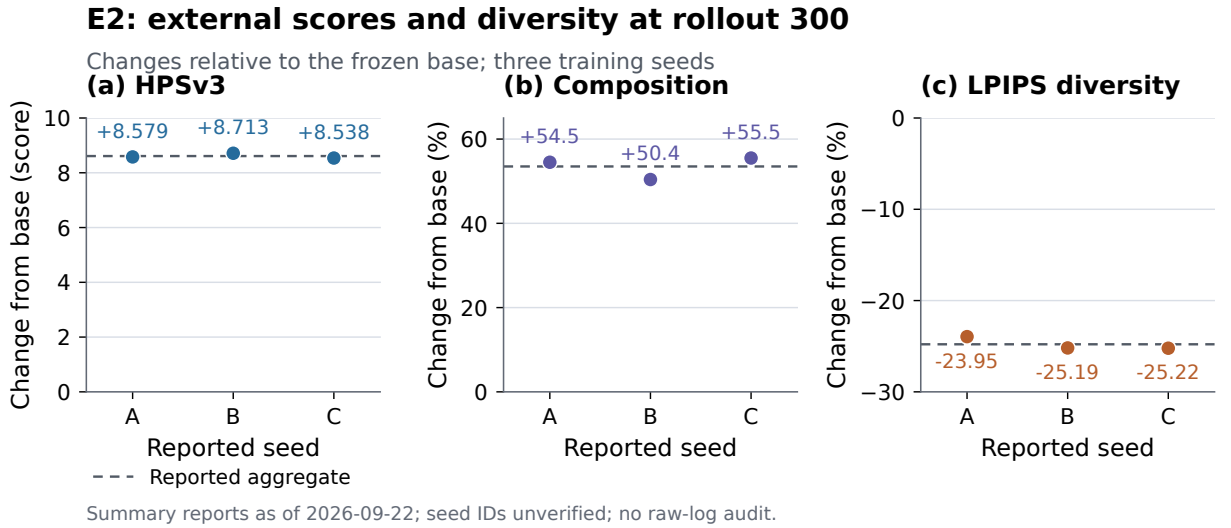


Figure 5: Reported per-seed E2 changes at the final checkpoint relative to the frozen base. Each panel retains its own metric and units; relative composition changes are not percentage-point changes. Points are training-seed summaries, not individual images. These are report-derived endpoints, with no raw-data confidence bands.

Reference comparison remains open. E2-R completed three training seeds and reports verified checkpoint exports, but no held-out head-to-head evaluation was performed. Export correctness cannot establish quality parity, and evaluation alone would not remove the native flow/CPS estimator mismatch. An explicitly labeled native-recipe comparison could provide context; strict reproduction still requires mathematical and effective-work parity.

HunyuanImage-3.0 recipe reproduction. We further evaluate UniRL on HunyuanImage-3.0-Instruct as a high-resolution text-to-image workload. The run is single-stage diffusion: the train-side recipe samples and replays only the image-generation mode, with no AR rollout or AR objective, so it does not exercise the shared-backbone AR and diffusion trajectory described in Sections 3.4 and 3.5. It uses one node with eight H20 GPUs, 16 prompts and four samples per prompt, producing 64 images per rollout at 1024×1024 resolution. Sampling uses 16 inference steps and three SDE steps, and training uses FlowGRPO with PickScore reward and seed 42. The exact resolved recipe is archived under `artifacts/reported_results/2026-09-30-hunyuanimage3/`.

We report the run through its durable checkpoint at rollout 360; the archived history continues to rollout 369. Mean training reward increased from **0.762** over the first ten rollouts to **0.870** over rollouts 351 through 360 (Figure 6). This result shows that the HunyuanImage-3.0 text-to-image recipe executes and improves its training reward in UniRL. Because the run uses one seed and has no held-out image-quality evaluation, we treat it as recipe-reproduction evidence rather than a comparative quality result.

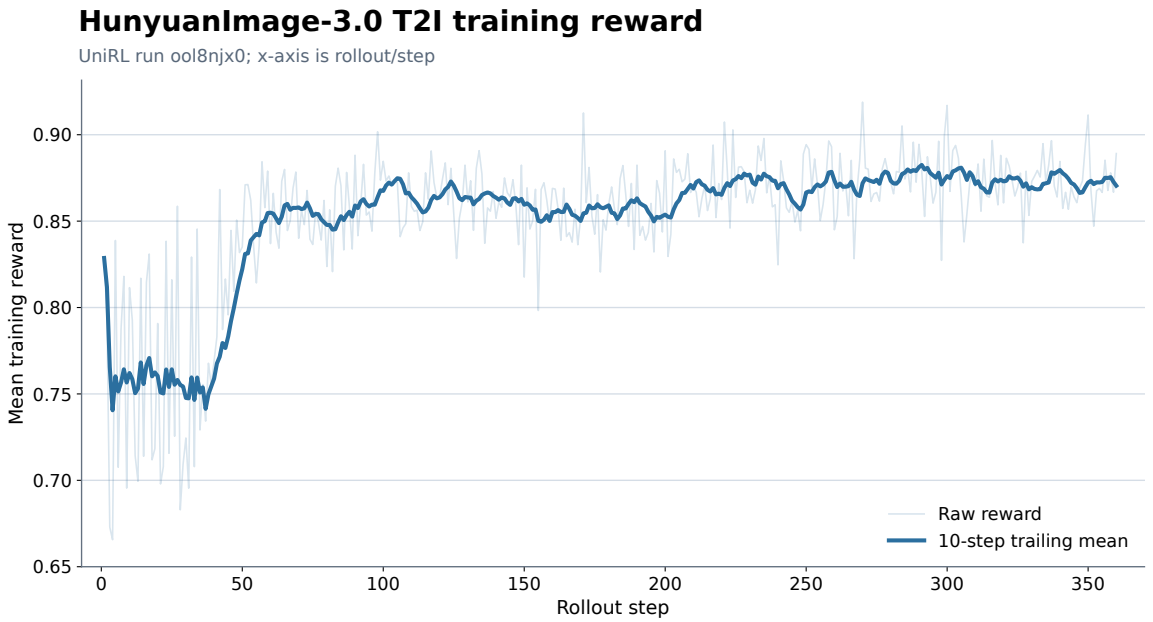


Figure 6: HunyuanImage-3.0 training reward in UniRL. The faint trace shows the reward at every rollout and the bold trace shows its trailing ten-rollout mean. The horizontal axis uses UniRL's logical `rollout/step`, rather than W&B's default `_step`.

MiniMax-H3 recipe reproduction. We also evaluate UniRL on MiniMax-H3 text-to-video-with-audio generation. Each logical step contains eight prompts with 16 samples per prompt, or 128 trajectories, generated at 256×384 resolution with 124 frames and ten denoising transitions. Training uses one optimizer update per logical step, learning rate 10^{-4}, ImageBind and CLAP rewards, seed 42, and one node with eight H20 GPUs. The ImageBind term scores audio–video correspondence on the middle frame and the CLAP term scores text–audio agreement; neither scores prompt–video alignment or temporal video quality. The resolved configuration is archived under `artifacts/reported_results/2026-09-30-minimax-h3/`.

We report the run through its durable checkpoint at logical step 200; the archived history continues to step 204. Mean reward increases from **0.194** over the first ten steps to **0.375** over steps 191 through 200 (Figure 7). Because this is a single-seed run with no held-out video and audio quality evaluation, we use it as evidence of executable recipe and model coverage rather than as a comparative-quality, systems-performance, or time-to-quality result.

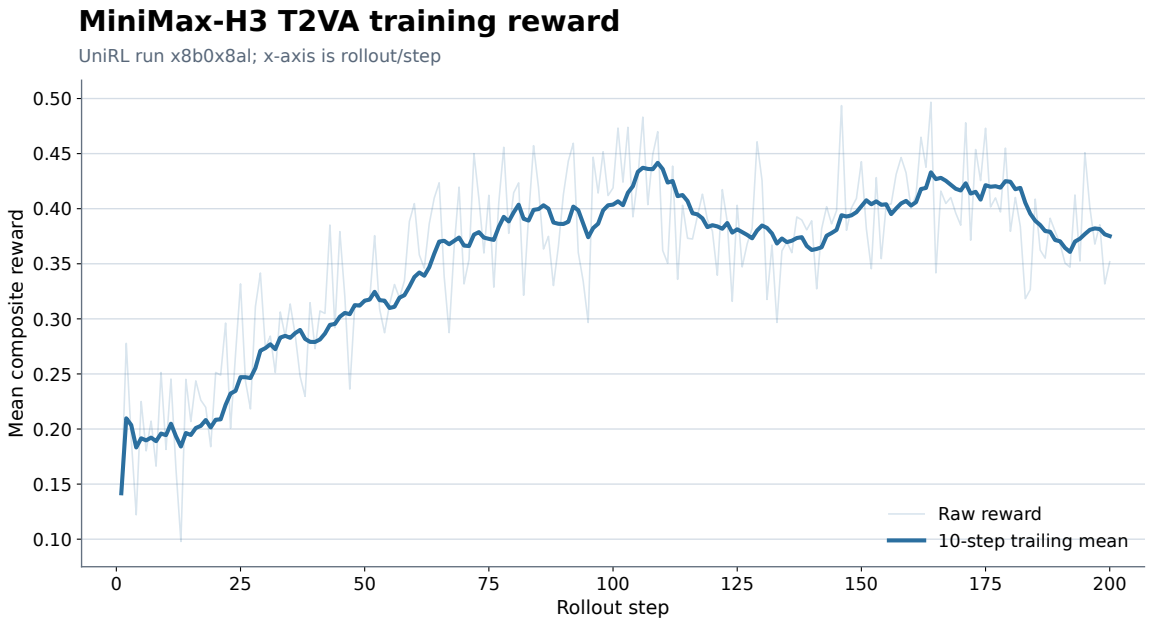


Figure 7: MiniMax-H3 training reward in UniRL. The faint trace shows the reward at every logical step and the bold trace shows its trailing ten-step mean. The horizontal axis uses UniRL's logical `rollout/step`, rather than W&B's default `_step`.

8.2 RQ2: What is end-to-end system performance?

We report two complementary performance comparisons. E3 compares UniRL with VeRL-Omni commit `01c87ee5` for the E2 SD3.5 shape-matched work unit on one reserved eight-GPU node (Huang et al., 2026); E3-AR is a separately rerun, recipe-matched Qwen3-4B GRPO comparison against veRL. The planned work unit uses 48 prompts, 16 samples, 384×384 output, ten denoising steps, three early-window SDE steps, two optimizer updates, microbatch eight, LoRA 32/64, learning rate/weight decay 10^{-4}, clip 10^{-5}, and PickScore. The later report identifies different realized LoRA target sets, so matching rank and alpha does not establish matched trainable parameter counts. The reported protocol uses three process-level repetitions per configuration, 30 steps each, removes the first five timing observations, and retains at least 20 observations per repetition. Systems are reported to run separately on the same reserved node; cache policy and randomized execution order remain checks against the underlying run records. We show the three distinct reported configurations, without duplicating UniRL as an additional "best valid" row. The stock UniRL recipe uses `FlowSDEStrategy` and three sampled early indices; the VeRL-Omni launcher requests `cps` and an early three-step window. These may change the transition distribution and likelihood, not merely kernel implementation. Normalization, anchors, actual replay work, and reward-service GPU placement also require an audit. At the audited release the stock SD3 recipes also enable update-membership shuffling (`shuffle_updates`), which the reported runs predate; a rerun at v0.2.0 must either disable it or report it as a further difference. Until a semantically aligned pair is established, E3 is labeled a *shape-matched native-recipe comparison*; attention-backend matching alone does not make it an equivalent-algorithm framework comparison. Its speed ratio cannot be attributed solely to framework overhead.

A recipe-matched AR systems comparison is reported below; the primary MATH-500 quality endpoint and a controlled time-to-quality comparison remain open.

The full protocol requests median, mean, and p90 iteration time, throughput, phase timing, memory, and utilization. The older report-derived E3 bundle supplies only selected aggregate timings and throughput, whereas E3-AR supplies the post-warmup per-step timing and response-length distribution but only one process-level run per framework. Strong and weak scaling use fixed global and fixed per-GPU work, re-

spectively, after the primary single-node pair is complete. The 30-step E3 protocol does not support a time-to-quality claim; the E2-R quality comparison is still missing and E6 did not complete a threshold-based study.

Table 11: Reported E3 native-recipe systems timings and E5-T deployment timings. All rows use eight allocated GPUs. E3 and E5-T answer different questions; differences in estimators and deployment prevent a pure framework or transport attribution. Aggregate values retain the source table's precision, not an independently established measurement accuracy.

System	Configuration	s/iteration	samples/GPU-h
E3: native recipes, eight GPUs in total			
UniRL	Native flow / SDPA-class	64.2	5381
VeRL-Omni	Native CPS / SDPA	111.3	3104
VeRL-Omni	Native CPS / FA3	112.3	3079
E5-T: deployment regimes, eight GPUs in total			
UniRL	Colocated, 8 shared GPUs	64.223	–
UniRL	Separate, 4 training + 4 rollout GPUs	146.244	–

E3 result and its comparison boundary. The reported UniRL and VeRL-Omni SDPA times are 64.22 and 111.33 seconds per iteration, a ratio of approximately 1.73. The observed SDPA and FA3 times are similar; this is not an equivalence test. The result concerns the complete measured native configurations, including their estimator and LoRA-target differences. The report also records greater short-run reward improvement per step for VeRL-Omni, so iteration throughput cannot be converted into a time-to-quality advantage. Neither the report's parameter-adjusted ratio nor its progress-normalized ratio is presented as an independently validated matched-control result. Generation phase labels are also non-equivalent: VeRL-Omni includes scoring in its generation timer, whereas UniRL separates it. We therefore omit a phase-breakdown figure until a common accounting boundary and the underlying measurements are available. The source PR body and manuscript disagree on some replicate summaries; the present figure uses only the explicitly identified aggregate table values and does not fabricate replicate points, confidence intervals, or statistical significance.

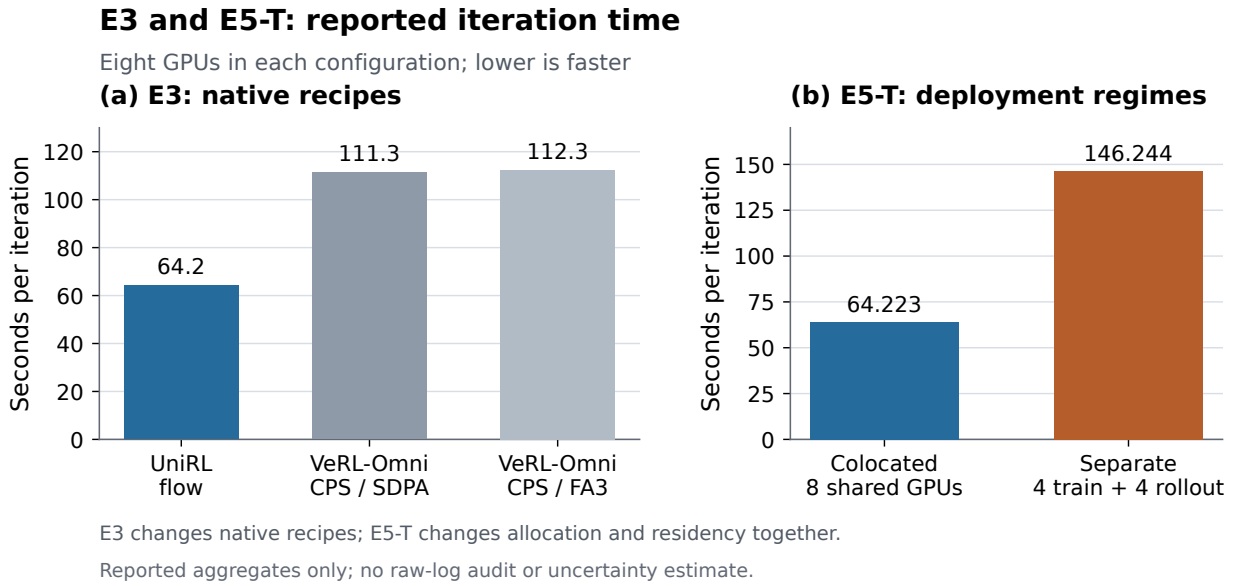


Figure 8: Reported iteration-time summaries. Left: E3 native-recipe comparison on one eight-GPU node. Right: E5-T's separate four-training/four-rollout allocation versus eight shared GPUs within UniRL. Lower is faster. Panels keep the external recipe comparison separate from the internal deployment comparison; no error bars are inferred from unavailable run-level data.

Recipe-matched autoregressive comparison. We additionally compare UniRL and veRL on a 500-step Qwen3-4B-Base (Yang et al., 2025) standard GRPO workload over DAPO-Math-17k (Yu et al., 2025), the algorithm the registered RQ1 protocol specifies. Each framework uses 16 NVIDIA H20 GPUs across two eight-GPU nodes. The matched recipe uses 64 prompts with eight samples per prompt, temperature and top-p 1.0, top-k disabled, natural EOS termination with an 8,192-token response cap, and four optimizer updates per rollout under a 10,240-token per-GPU optimizer budget. The two runs align the model, dataset and seeded prompt shuffle, tokenizer, verifier, GRPO objective (advantage divided by the group standard deviation, per-sequence token-mean loss, symmetric clip $\epsilon = 0.2$), rollout log-probabilities as the frozen π_{old} anchor, optimizer settings, fp32 master weights with bf16 compute, rollout batch, and update count. UniRL uses eight SGLang TP=2 engines, while veRL uses eight asynchronous vLLM TP=2 engines; these serving stacks are part of the compared end-to-end systems. The recipes do not align the training attention kernel (UniRL uses FlexAttention), reward placement (UniRL scores responses in a separate reward stage, whereas veRL's reward manager runs inside rollout), or realized response lengths, which follow each policy's own training trajectory. We exclude startup step 1 and report steps 2–500.

Table 12: Recipe-matched Qwen3-4B GRPO performance over steps 2–500. Recipes are matched; realized response lengths are not. Times and response length are mean ± step-level population standard deviation over 499 steps. Token rates divide total generated tokens by total measured time. Step-level deviations describe these runs and are not process-level uncertainty estimates.

System	E2E (s/step)	Generation (s/step)	Training (s/step)	Response tokens	E2E tok/s	Generation tok/s
UniRL	143.519 ± 28.601	**76.059 ± 15.129**	61.196 ± 13.458	4,197.6 ± 1,005.4	**14,975**	**28,257**
veRL	**139.894 ± 30.731**	80.619 ± 16.308	**55.743 ± 14.577**	3,781.8 ± 1,039.1	13,841	24,018

AR systems result. UniRL's mean raw end-to-end step time is 2.6% higher than veRL's while it samples responses that are 11.0% longer on average. The median raw end-to-end step time is 147.7 s for UniRL and 150.1 s for veRL, and the 90th percentiles are 172.6 s and 169.6 s. After accounting for this varying work, UniRL provides 8.2% higher end-to-end output-token throughput and 17.7% higher generation output-token

throughput. The native veRL actor-update timer is lower than the UniRL training timer, whereas UniRL's generation advantage is larger for this workload. The raw end-to-end ordering reverses near step 300 as the two sampled response-length distributions cross, but UniRL's token-normalized end-to-end and generation throughput are higher in every one of the five 100-step bands. We therefore treat token-normalized end-to-end throughput as the primary result rather than claiming a latency advantage at every training point. Phase values use each framework's native instrumentation and are diagnostic; end-to-end time has the clearest common boundary. Two estimator-level differences remain that configuration cannot remove: UniRL divides advantages by the population standard deviation and veRL by the sample standard deviation, making veRL's advantages uniformly $\sqrt{7/8} = 0.935\times$ smaller at eight samples per prompt, and the two advantage epsilons differ by 10^{-8} versus 10^{-6}; both are quantified against their closed forms in the bundle's audit. This experiment contains one process-level run per system and does not establish repeated-run uncertainty or time-to-quality.

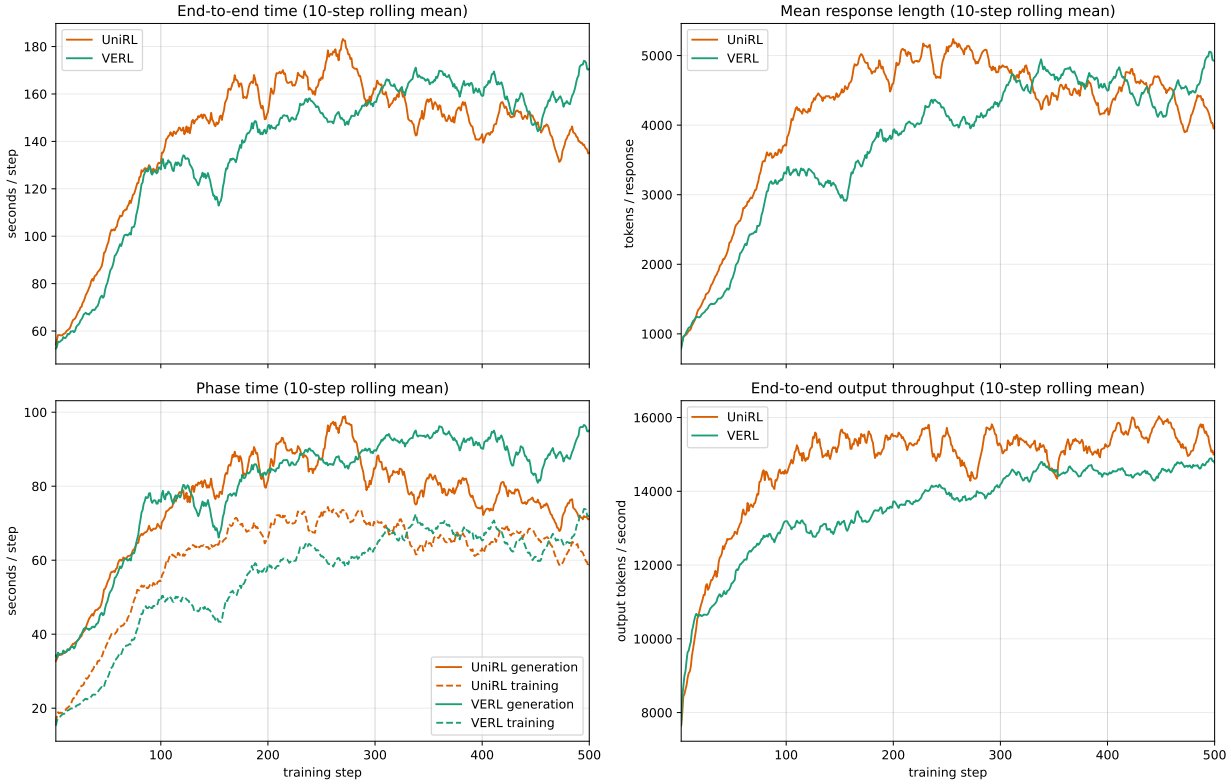


Figure 9: Ten-step rolling means for the 500-step natural-length Qwen3-4B GRPO comparison. Step 1 is excluded as startup warm-up. Response lengths evolve during training and cross near step 300, reversing the raw step-time ordering; token-normalized end-to-end throughput remains higher for UniRL throughout the five reported bands.

8.3 RQ3: What does the trajectory abstraction cost and enable?

RQ3 first tests the semantic capability that motivates the IR, then measures its cost. E4 uses the executable frozen-Qwen3-0.6B rewrite → SD3.5 path: eight original prompts, four rewrites per prompt, and eight images per rewrite, or 256 image descendants per rollout; only diffusion trains. This is a separate recipe from E2: 512×512, ten steps, one sampled SDE index among 0–8, $\eta = 0.7$, LoRA 16/32, learning rate 3×10^{-4}, weight decay zero, clip 10^{-4}, replay anchor, two updates, and microbatch one. The primary condition (`diffusion_group_scope=prompt`) computes group-relative advantages across all 32 descendants of one original prompt, while `diffusion_group_scope=rewrite` (the trainer default) groups the eight images under each rewrite. The two stock prompt-enhancement recipes are not a matched pair: the prompt-group

recipe uses eight images per rewrite and a 512-token rewrite limit, whereas the rewrite-scope recipe uses four and 256. Both arms must therefore use the prompt-group recipe with only the grouping key changed, as confirmed by their resolved configurations; both stock recipes also enable `shuffle_updates` at the audited release. Both conditions must serialize redacted root/part/parent IDs, shapes, reward/advantage, replay indices, and group labels; the synchronous prompt-enhancement trainer does not advance output versions, so they carry no behavior-version provenance here. Invalid lineage remains an E0 fail-fast test rather than an intentionally corrupted quality baseline.

For image $x_{p,j,k}$, the two group keys are $g_{\text{root}} = p$ and $g_{\text{rewrite}} = (p, j)$. The intended semantic test requires an ID-joined flat-row reference to reproduce UniRL's group memberships, advantages, and learner inputs on identical recorded rewards and replay tensors, including after root-shard round trips. The 300-rollout, three-seed training comparison changes the normalization population (32 versus eight images); any quality difference would therefore concern the grouping objective, not isolate the IR representation's effect.

Passing that recorded-trajectory control would establish the grouping semantics on real rollout data; it would not by itself prove better task quality. The current PickScore request is conditioned by the rewritten text (the default `prompt_source: generation`; scoring against the root prompt with `prompt_source: original` is available but was not used in E4), so a claim about preserving the user's original intent additionally requires lineage-keyed evaluation of every image against its root prompt. We therefore report both rewrite-conditioned training reward and frozen HPSv3/ImageReward evaluations joined to each root prompt. Evaluation generates held-out images at 512×512, 40 steps, and guidance 1 with the base and every 50-rollout diffusion checkpoint, using one fixed manifest of four frozen rewrites per root and eight image seeds per rewrite for both conditions. Worker/adapter RNG initialization, deterministic trainside-AR sampling, real trace dumping, and this held-out generation/evaluation path are E0 prerequisites for a multi-seed E4 result. If the two correct groupings tie, the experiment still establishes semantic expressiveness but not a quality benefit.

E4 reported results and unresolved input validity. E4a reports 24/24 successful checks in each scope and bit-exact agreement with an ID join. The fixtures are synthetic, so this is bounded semantic evidence rather than the recorded-rollout check specified above. E4b reports no consistent advantage across three seeds: the differences change sign. This does not demonstrate equivalence or establish that the effect is zero. The PR additionally discloses malformed rewrite conditioning: chat markers remain in many outputs and many rewrites exceed the CLIP token window because the intended extraction configuration was not applied. Those defects can affect both absolute quality and the grouping contrast; a statistically inconclusive outcome does not make them irrelevant. The measured configuration is retained as an exploratory result, while a corrected, separately versioned pipeline must pass a short health check before further quality comparisons.

E5-R remains unmeasured. The intended microbenchmarks separate recursive IR operations and tensor transport from model compute, varying root count, branch factor, tensor payload, token/media lengths, and part depth. Required outputs include structural-operation time, metadata bytes, dense bytes avoided at the driver, transfer time, and reassembly correctness hashes.

The broader E5 protocol compares the colocated store, per-GPU store, and TransferQueue path where hardware permits. The current report confirms that the available harness does not implement the required paired representation measurement. The missing comparison is whole-tree sharding versus explicit ID-joined flat rows on the same recorded payload, device, transport, and operations. This paired representation slice holds model work, device placement, values, grouping, and replay inputs fixed; driver RSS, bytes moved, operation time, and fraction of iteration cost measure the price of preserving structure. Semantic errors are caught in E0 and are never timed as valid work.

The minimum topology slice reuses E3's colocated SD3.5 UniRL row and adds the same work with `sd3_vllmomni_lora_separate.yaml`: four GPUs train and four generate while the total eight-GPU allocation remains fixed. That recipe carries the stock rather than the E3-aligned settings and the repository has no aligned launcher for it, so the E3 overrides must be applied and archived with the separate arm. This pair measures two deployment regimes, not a pure transport primitive—training parallelism, residency, and local versus remote LoRA publication change together and require a common phase breakdown be-

fore attributing their costs. Broader same-/cross-GPU and cross-node backend measurements remain E5 extensions. The reported E5-T times are 64.223 seconds for shared deployment and 146.244 seconds for the separate allocation, approximately 2.277 times longer (Figure 8). The separate UniRL configuration is also slower than the colocated VeRL-Omni E3 configuration. The report attributes much of the increase to reward processing, but without the phase records we retain this as a reported diagnosis, not an isolated causal result. Neither this deployment penalty nor the CPU timings below establish bounded representation overhead. We inventory which bundle/stage/adapter code the E4 path adds and which trainer, scheduler, transport, and credit modules it reuses unchanged, as a concrete instance of the $M + N$ composition in Section 3.6. **TODO[data]:** Complete the E4 reuse inventory (added versus unchanged modules, with line counts at the experiment commit) and attach it as an artifact before release.

Preliminary executable evidence. We ran a source-importing regression harness against a clean UniRL checkout at `fd344ad`; it does not mock the trajectory classes. All ten checks passed: cross-stage lineage construction, exact tree split/concat, whole-tree select/slice, leaf-to-root reward propagation and group advantages, rejection of malformed lineage and primitive modality/batch violations, variable-length packed-text round trips, and lazy `TensorRef` selection/materialization including bounds checking. Table 13 reports a deliberately narrow timing slice from the same run. The synthetic workload has four AR children per root, one observation per AR child, two diffusion children per observation, and a four-value image payload per leaf.

Table 13: Preliminary single-process CPU trajectory-IR timings (microseconds). Python 3.12.13 and PyTorch 2.11.0; one thread; ten warm-up and 100 measured repetitions. These are regression measurements, not distributed or GPU throughput.

Roots	Leaf rows	Operation	Median	p90
8	64	`sample.split_concat`	120.8	142.6
8	64	`sample.select_half`	556.8	616.7
8	64	`tensorref.select_strided`	12.3	13.0
32	256	`sample.split_concat`	291.2	329.6
32	256	`sample.select_half`	2231.5	3126.8
32	256	`tensorref.select_strided`	47.0	52.4
128	1024	`sample.split_concat`	1435.5	2231.7
128	1024	`sample.select_half`	8152.4	8458.7
128	1024	`tensorref.select_strided`	202.2	226.4

The current `Sample.select` path forms tree-complete root shards before gathering them, so its cost grows with the full input tree rather than only the selected rows. Strided `TensorRef` selection creates many metadata spans while contiguous selection coalesces them; no dense tensor is copied until materialization. These observations motivate, but do not replace, the planned GPU transport and end-to-end ablations. The manifest, 100 per-cell raw trials, structured check outcomes, generated table rows, and SHA-256 checksums are archived under `artifacts/trajectory_ir/cpu_fd344ad_macos_arm64/`.

8.4 RQ4: Exploratory bounded-staleness observations

The prespecified study separated the allocation contrast (colocated synchronous S0 versus disaggregated D0 without overlap) from the onset of overlap (D0 versus A1), followed by publication/lag sweeps. The report states that the formal sweep was canceled on 2026-09-21; only single-seed capacity measurements and a separate quality-target pilot were completed. The capacity timings, obtained with evaluation disabled, are planning observations rather than a full-budget, equal-quality comparison and are not used for a headline speedup.

The quality-target pilot did not pass. The D0 pilot's reported MATH-500 avg@4 decreased from 0.798 at rollout 150 to 0.773 at rollout 300, contrary to the preregistered requirement that the later checkpoint improve. No quality threshold was registered and no time-to-quality result is reported. Training reward also declined over the later window. This is a negative outcome for the tested pilot criterion; it neither identifies asynchrony as the cause nor establishes that every bounded-staleness configuration is ineffective. Two checkpoints are insufficient to locate a precise turning point.

Evaluation variability and requirements for a restart. The report's repeated evaluation of one checkpoint found variable prompt-level avg@4 scores despite a relatively small aggregate difference. That observation is specific to the sampled checkpoint and decoding protocol; it is not a universal noise floor or a substitute for training-seed replication. A renewed study would first audit scorer behavior, request-level sampling and repeat-evaluation variability; match the synchronous/asynchronous estimator and learner settings; and record complete driver time and buffer admission, rejection, discard, retry, and completion accounting. The source-audited `perf/step_time_s` alone omits outer-loop publication, evaluation, and checkpoint work. Only after establishing a suitable learning budget and a separately piloted target should a replicated formal grid support a lag–quality–time conclusion. The original grid and preflight controls remain documented in `GPU_HANDOFF_README.md`; they are not represented here as completed experiments.

8.5 Fairness and statistical protocol

Effective work is matched before wall-clock time: prompts, samples per prompt, decoded resolution or maximum tokens, rollout steps, replayed steps/tokens, microbatching semantics, and optimizer updates. Primary full-system metrics count all allocated training, rollout, and reward resources on both sides. Reward placement and concurrency must be disclosed; a reward-excluded number is supplementary and carries a separate service-cost report. We prespecify warmup removal, use at least 20 steady-state iterations for throughput summaries, and report the full distribution rather than a single minimum. Training summaries use the seed as the statistical unit. The present figures show the available individual-seed summaries; a full release should additionally report justified seed-level uncertainty. Bootstrap intervals over prompts or images must be labeled separately and cannot replace seed uncertainty. The complete systems release must show all process repetitions and the post-warmup per-step distribution; uncertainty of a speed ratio must use repetitions, not correlated steps, as its independent units. E3-AR releases its 499-step post-warmup distribution, but has only one process-level run per framework; the older report-derived E3 summaries provide neither the complete per-step distribution nor independently revalidated repetitions. The final checkpoint and named primary endpoint are fixed in advance; the auxiliary metrics diagnose failure modes and are not alternative endpoints chosen after seeing results. Any future E6 time-to-quality study must use MATH-500 and a target registered from a separate pilot before the formal grid, bracket first observed crossings by evaluation times, and censor runs that never cross. The canceled study produced no such result. Elapsed time spans initialization, training, publication, checkpointing, and evaluation; steady-state throughput is reported separately. All published cells require complete scoring or an explicit missingness analysis; silently averaging only successful evaluator responses is inadmissible.

9 Related Work

LLM post-training systems. HybridFlow/veRL combines single- and multi-controller execution and introduces hierarchical APIs for mapping RLHF dataflows to devices (Sheng et al., 2024). OpenRLHF uses Ray, vLLM, and DeepSpeed to schedule the models in an RLHF workflow (Hu et al., 2024). ROLL emphasizes a single-controller abstraction, parallel workers, sample lifecycle scheduling, and flexible device mapping (Wang et al., 2025). NeMo-RL provides scalable training and generation backends and multimodal model support (NVIDIA Corporation, 2025). Group-relative recipes in these systems (Sheng et al., 2024; Hu et al., 2024; Wang et al., 2025; NVIDIA Corporation, 2025) typically record one level of sampling structure—responses grouped under their prompt by a per-prompt identifier or by contiguous sample order—whereas UniRL generalizes that group key to multi-level ancestry encoded in sample identifiers. These systems already drive distinct generation and training backends from one control loop and colocate or separate those backends as a placement choice; UniRL keeps that split and changes the object the loop schedules. Scheduling and data-parallel sharding move complete root groups rather than rows, and each generated layer carries the replay segment for its own AR or diffusion stage (Section 3). A flat-row batch can recover the same group memberships by joining on retained parent keys (Section 2); UniRL differs in that sharding, credit assignment, and cross-stage conditioning consume that ancestry in place, so changing the execution path does not require rebuilding the join. RQ3 specifies the corresponding control: an ID-joined reference on recorded rewards and replay tensors (Section 8.3). To date its agreement check uses synthetic fixtures, and

the paired cost of carrying the structure (E5-R) is unmeasured; the single-process CPU regression is not that measurement.

Asynchronous post-training. AReaL fully decouples generation and training and controls staleness to improve language-reasoning throughput (Fu et al., 2025). Relax uses service-level separation, a TransferQueue data bus, and configurable staleness for omni-modal post-training (Zhang et al., 2026). slime supports persistent asynchronous rollout generation around a training/rollout/data-buffer loop (Zhu et al., 2025). These systems make asynchrony an execution design space. UniRL takes a conservative point within it: publication remains a barrier, and lag is bounded in optimizer-update units (Section 4.5). Each generated part carries its output policy version in the trajectory IR, while admission, lag filtering, and publication stay in one manager shared by the AR and diffusion trainers. The filter keeps or drops whole root groups so that group-relative baselines stay complete, and no training batch spans behavior versions; this is stricter than AReaL, whose decoupled objective admits trajectories generated under several policy versions (Fu et al., 2025). RQ4 reports only single-seed capacity measurements and a quality-target pilot that did not pass; it supports no throughput or time-to-quality comparison with these systems (Section 8.4).

Multimodal and diffusion post-training systems. VeRL-Omni extends veRL toward diffusion, unified, and omni-modality models and integrates vLLM-Omni rollout (Huang et al., 2026). Relax is explicitly omni-native (Zhang et al., 2026). Modality coverage therefore does not distinguish UniRL from these systems. The distinction is how ancestry and typed replay state survive execution: an AR stage and a diffusion stage are two layers of one `Sample` (Section 3), and tensor references move their dense state without changing that lineage (Section 4). Models and algorithms meet only at the per-stage conditions, segment, and advantages that an algorithm consumes, so M models and N shared algorithms require $M + N$ rather than $M \times N$ integrations (Section 3.6). E4 runs one frozen AR rewrite stage into a trained diffusion stage (Section 8.3). Its lineage check uses synthetic fixtures, its quality comparison is inconclusive across three seeds, and its rewrite text was malformed; the pending module inventory would list what that path reuses, not measure integration effort.

Diffusion RL algorithms. DDPO casts denoising as a multi-step decision process (Black et al., 2023). Flow-GRPO converts flow sampling to an SDE and applies group-relative optimization (Liu et al., 2025); DanceGRPO generalizes related ideas across visual generation tasks (Xue et al., 2025); DiffusionNFT instead optimizes a forward-process objective from reward-ranked generations (Zheng et al., 2025). UniRL consumes FlowGRPO-style and DiffusionNFT objectives through its algorithm interface and claims neither objective (Section 7). What the interface contributes is that their incompatible sufficient statistics stay explicit: FlowGRPO-style objectives replay recorded SDE transitions, DiffusionNFT trains on clean generated samples, and both reuse the same train stack for microbatching and optimizer state.

10 Limitations and Open Evidence

Integration cost is per architecture. Adding an architecture requires one bundle with its stages, conditions, and rollout adapters; this cost is paid once per model rather than once per model–algorithm pair. Architecture-specific details such as tokenizers, noise schedulers, latent layouts, and LoRA targets stay inside that bundle and its adapters rather than being abstracted by the trajectory type. The ordered-layer representation targets trajectories that grow by forking and observation; arbitrary ragged forests are outside its current scope.

Topology is configured, not optimized. Placement scopes make resource choices explicit but do not search for the best split. The best topology depends on model memory, reward cost, rollout variance, interconnect, and publication size.

Asynchronous execution is bounded and partially specialized. Publication is a barrier, and current AR and diffusion paths impose different admissible in-flight structure. Fault isolation is limited by the shared Ray runtime and role lifecycle; the system does not yet claim service-level recovery comparable to dedicated service architectures.

The empirical case is incomplete. The GPU evidence supports qualified observations about diffusion scores and diversity, native-recipe iteration time, deployment sensitivity, and one recipe-matched AR throughput comparison. The AR comparison releases per-step evidence but lacks independent process repetitions and a common external quality endpoint; it therefore does not establish run-to-run uncertainty, training-quality reproduction, or time-to-quality. The combined evidence also does not establish a quality advantage from lineage grouping, bounded IR overhead, scaling, or a time-to-quality benefit from asynchrony. Raw GPU records remain unavailable for the legacy report-derived studies, which still need exact run manifests, consistent aggregate provenance, and independent regeneration. The malformed rewrite pipeline and synthetic-only E4a fixtures further limit the evidence for the central cross-stage claim.

11 Conclusion

UniRL treats post-training as a trajectory program whose semantic object survives changes in generative family and execution topology. Lineage, typed modality values, replay segments, reward, credit, and policy version live in one IR; rollout, reward, training, transport, and publication are logical roles mapped separately to resources. This design is most valuable if it passes two empirical tests: credible learning behavior across representative AR and diffusion workloads, and competitive end-to-end execution without weakening semantic parity. Those tests are necessary but not sufficient for the paper's trajectory-centric thesis: a real composed run must also demonstrate lineage-defined credit/grouping, and the measured IR/topology costs must bound the price of preserving that structure. The current evidence shows diffusion score improvements accompanied by reduced diversity, faster iteration of one native diffusion configuration, a substantial penalty for the measured separate deployment, and 8.2% higher end-to-end output-token throughput for UniRL in one recipe-matched AR systems run. That AR result is a workload-specific throughput observation rather than a repeated-run or time-to-quality conclusion. The full trajectory-centric claim remains open until real-rollout lineage validation and matched representation-cost measurements complement those observations. Bounded staleness remains an exploratory operating-region question, with no demonstrated time-to-quality benefit in the canceled study.

Reproducibility statement

The source characterization in this draft refers to UniRL release v0.2.0, commit `fd344ad6036a0ba7be9900a9d0cf78ebf59fa738` (2026-10-08). The manuscript repository records the claim-to-evidence plan in `PAPER_PLAN.md`. The cluster-ready execution protocol, configuration-parity gates, metric inventory, and immutable run-directory contract are specified in `EXPERIMENT_RUNBOOK.md`; the command-oriented transition to a fresh GPU-side agent is in `GPU_HANDOFF_README.md`. Both documents intentionally contain no results. The current result audit is in `EXPERIMENT_RESULTS.md`. The dated bundle `artifacts/reported_results/2026-09-22/` stores compact source snapshots, source-level caveats, reported numerical inputs, and generated tables and figures. These reproduce the transcription and presentation of the reports, not the original GPU measurements. The separately rerun AR systems bundle in `artifacts/reported_results/2026-10-11-qwen3-4b-grpo/` pins UniRL commit `6148a5d`, the veRL reference and compatibility commits, both resolved recipes, the recipe and effective-work audit, run identities, 499 post-warmup records per framework, phase and band summaries, and the plotting script. Commit `6148a5d` is on a pre-release branch: it descends from an earlier audit commit but carries four rollout-performance and decoding commits that v0.2.0 does not contain, so E3-AR measures that branch rather than the audited release. Its UniRL run is W&B run `2x4tdjwa`; its veRL run is `grpo-verl-natural500`, whose per-step records come from the trainer's own console metrics. The HunyuanImage-3.0 and MiniMax-H3 bundles in `artifacts/reported_results/2026-09-30-hunyuanimage3/` and `artifacts/reported_results/2026-09-30-minimax-h3/` store the resolved recipes, W&B run identities, direct per-step history exports, and rendered learning curves. All graphics are committed PDF assets included by relative paths, so Overleaf does not require Python or shell escape to build the manuscript. The preliminary RQ3 evidence is regenerated by `experiments/trajectory_ir/run_cpu_evidence.py`; its manifest pins the source state, command, environment, workload, warm-up, repetitions, and timer. **TODO[data]:** Before release, reconcile the legacy E3 replicate summaries and E2 evaluation protocol,

repeat E3-AR at the process level, complete the primary AR quality and core IR evidence, and add author metadata and a durable artifact URL.

References

Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. 2023. URL https://arxiv.org/abs/2305.13301.

Chaorui Deng, Deyao Zhu, Kunchang Li, Chenhui Gou, Feng Li, Zeyu Wang, Shu Zhong, Weihao Yu, Xiaonan Nie, Ziang Song, Guang Shi, and Haoqi Fan. Emerging properties in unified multimodal pretraining. 2025. URL https://arxiv.org/abs/2505.14683.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. 2025. URL https://arxiv.org/abs/2505.24298.

Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. 2024. URL https://arxiv.org/abs/2405.11143.

Yongxiang Huang, Cheung Kawai, Jingan Zhou, Yingshu Chen, openYuanrong Team, and Xibin Wu. VeRL-Omni: Easy, fast, and stable RL training for diffusion and omni-modality models. GitHub repository, 2026. URL https://github.com/verl-project/verl-omni.

Amita Kamath, Kai-Wei Chang, Ranjay Krishna, Luke Zettlemoyer, Yushi Hu, and Marjan Ghazvininejad. GenEval 2: Addressing benchmark drift in text-to-image evaluation. 2025. URL https://arxiv.org/abs/2512.16853.

Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. 2023. URL https://arxiv.org/abs/2305.01569.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL https://arxiv.org/abs/2309.06180.

Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online RL. 2025. URL https://arxiv.org/abs/2505.05470.

Xin Luo, Jiahao Wang, Chenyuan Wu, Shitao Xiao, Xiyan Jiang, Defu Lian, Jiajun Zhang, Dong Liu, and Zheng Liu. EditScore: Unlocking online RL for image editing via high-fidelity reward modeling. 2025. URL https://arxiv.org/abs/2509.23909.

Qianli Ma, Yaowei Zheng, Zhelun Shi, Zhongkai Zhao, Bin Jia, Ziyue Huang, Zhiqi Lin, Youjie Li, Jiacheng Yang, Yanghua Peng, Zhi Zhang, and Xin Liu. VeOmni: Scaling any modality model training with model-centric distributed recipe zoo. 2025a. URL https://arxiv.org/abs/2508.02317.

Yuhang Ma, Yunhao Shui, Xiaoshi Wu, Keqiang Sun, and Hongsheng Li. HPSv3: Towards wide-spectrum human preference score. 2025b. URL https://arxiv.org/abs/2508.03789.

Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation*, 2018. URL https://www.usenix.org/conference/osdi18/presentation/moritz.

NVIDIA Corporation. NVIDIA multi-process service. Documentation. URL https://docs.nvidia.com/deploy/mps/latest/.

NVIDIA Corporation. NeMo RL: A scalable and efficient post-training library. GitHub repository, 2025. URL https://github.com/NVIDIA-NeMo/RL.

Ruoyu Qin et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. 2024. URL https://arxiv.org/abs/2407.00079.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021. URL https://arxiv.org/abs/2103.00020.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017. URL https://arxiv.org/abs/1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. 2024. URL https://arxiv.org/abs/2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. 2024. URL https://arxiv.org/abs/2409.19256.

Stability AI. Stable diffusion 3.5 medium model card. Hugging Face model repository, 2024. URL https://huggingface.co/stabilityai/stable-diffusion-3.5-medium.

Weixun Wang et al. ROLL: An efficient and user-friendly scaling library for reinforcement learning with large language models. 2025. URL https://arxiv.org/abs/2506.06122.

Keming Wu, Sicong Jiang, Max Ku, Ping Nie, Minghao Liu, and Wenhu Chen. EditReward: A human-aligned reward model for instruction-guided image editing. 2025. URL https://arxiv.org/abs/2509.26346.

Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. 2023. URL https://arxiv.org/abs/2306.09341.

Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and evaluating human preferences for text-to-image generation. 2023. URL https://arxiv.org/abs/2304.05977.

Zeyue Xue, Jie Wu, Yu Gao, Fangyuan Kong, Lingting Zhu, Mengzhao Chen, Zhiheng Liu, Wei Liu, Qiushan Guo, Weilin Huang, and Ping Luo. DanceGRPO: Unleashing GRPO on visual generation. 2025. URL https://arxiv.org/abs/2505.07818.

An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. 2025. URL https://arxiv.org/abs/2505.09388.

Peiqi Yin, Jiangyun Zhu, Han Gao, Chenguang Zheng, Yongxiang Huang, Taichang Zhou, Ruirui Yang, Weizhi Liu, Weiqing Chen, Canlin Guo, Didan Deng, Zifeng Mo, Cong Wang, James Cheng, Roger Wang, and Hongsheng Liu. vLLM-Omni: Fully disaggregated serving for any-to-any multimodal models. 2026. URL https://arxiv.org/abs/2602.02204.

Qiying Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. 2025. URL https://arxiv.org/abs/2503.14476.

Liujie Zhang, Benzhe Ning, Rui Yang, Xiaoyan Yu, Jiaxing Li, Lumeng Wu, Jia Liu, Minghao Li, Weihang Chen, Weiqi Hu, and Lei Zhang. Relax: An asynchronous reinforcement learning engine for omni-modal post-training at scale. 2026. URL https://arxiv.org/abs/2604.11554.

Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. URL https://arxiv.org/abs/1801.03924.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Bali, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. 2023. URL https://arxiv.org/abs/2304.11277.

Kaiwen Zheng, Huayu Chen, Haotian Ye, Haoxiang Wang, Qinsheng Zhang, Kai Jiang, Hang Su, Stefano Ermon, Jun Zhu, and Ming-Yu Liu. DiffusionNFT: Online diffusion reinforcement with forward process. 2025. URL https://arxiv.org/abs/2509.16117.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, and Joseph E. Gonzalez. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2312.07104.

Zilin Zhu, Chengxing Xie, Xin Lv, and contributors. slime: An LLM post-training framework for RL scaling. GitHub repository, 2025. URL https://github.com/THUDM/slime.

A Claim-to-source audit

Table 14 records the executable source used to support mechanism descriptions. It does not prove performance or successful execution for every configuration.

Table 14: Primary code evidence at commit `fd344ad` (v0.2.0).

Mechanism	Primary source symbols
Trajectory IR	`unirl/types/sample.py`: `Part`, `Sample`; `unirl/types/primitives.py`; `unirl/types/segments/`
Batch algebra	`unirl/distributed/tensor/batch.py`: `FieldKind`, `Batch`
Tensor references	`unirl/distributed/tensor/ref.py`: `TensorSpan`, `TensorRef`; `unirl/distributed/tensor/transport.py`: `TensorTransport`
Roles and placement	`unirl/distributed/group/remote.py`: `Remote`; `device_pool.py`, `placement.py`, `handle.py`, `dispatch.py`
Rollout boundary	`unirl/rollout/engine/base.py`: `BaseRolloutEngine`; concrete engines under `unirl/rollout/engine/`, including `vllm/`; `unirl/reward/stack.py` for overlapped scoring
Training boundary	`unirl/algorithms/base.py`: `StageAlgorithm`; `unirl/train/stack/base.py`: `TrainStack`; `anchor.py`, `planner/update.py`
Sync trainers	`unirl/trainer/ar.py`: `ARTrainer`; `unirl/trainer/diffusion.py`: `DiffusionTrainer`; `pe.py`; `residency.py`
Async and agentic	`unirl/trainer/async_rollout.py`, `async_ar.py`, `async_diffusion.py`, `agentic.py`; `unirl/rollout/manager/`
Weight publication	`unirl/distributed/weight_sync/`, including `full/ckpt_engine_ipc.py`; engine-specific receivers under `unirl/rollout/engine/`
Instrumentation	`unirl/utils/profiling.py`, `memory_monitor.py`, `wandb_logger.py`

B Colocated residency orders

Table 15 lists the orders that the trainers at the audited release use on a colocated slab. Each satisfies the two invariants of Section 4.3: a due publication completes before generation, and memory is released before the role that needs it is brought up.

Table 15: Colocated residency orders. "Offload" moves training state to host memory; "sleep" releases the engine's weight and cache memory, and "wake" restores it.

Order	Applies when	Before generation	After generation
Default	AR, agentic, and prompt-enhancement trainers, unless a row below applies	Wake the engine, publish if due, offload training state if enabled	Sleep the engine, then restore training state
Split wake	AR; publication due, offload enabled, engine can restore weight memory separately	Wake weight memory only, publish, offload training state, complete the wake	As default
Split wake, vLLM weight transfer	As above, publishing through vLLM's native weight-transfer engine	Offload optimizer state, wake weight memory, publish, offload the model shard, complete the wake	As default
Offload without publication	AR; offload enabled, no publication due	Offload training state, then wake the engine	As default
Extract, then push	AR with the engine pinned to one designated GPU of the slab; unified-model trainer with separate AR and diffusion engines	Extract weights if due, offload training state if enabled, wake the engine(s), push the extracted weights	As default; a pinned AR engine with offload disabled stays awake
Per-role residency	Diffusion trainers	Extract the adapter first if training will be offloaded; offload every non-resident role, wake the engine, publish if due	Sleep the engine unless the rollout role is resident; restore training state before the optimizer step

C Required run artifact schema

Every result row must resolve to a run directory containing:

- source and baseline commit hashes, submodule hashes, and dirty patches;
- the launch command and fully resolved configuration;
- model, reward, dataset, and tokenizer revisions or content hashes;
- GPU model/count, node/interconnect description, driver, CUDA, PyTorch, generation-engine, and training-backend versions;
- raw stdout/stderr, per-step structured timing, memory and utilization traces, checkpoint metadata, and evaluation outputs;
- seed, warmup exclusion rule, failure/retry accounting, and statistical aggregation script.

For training runs, serialized trace samples must redact private data while retaining lineage IDs, shapes, dtypes, policy versions, reward components, and segment index metadata. For system baselines, an alignment manifest must explain every nonidentical setting and whether it changes effective work.

D Correctness status and remaining checks

Executed locally. The preliminary RQ3 harness performs ten fail-fast checks on the audited source: lineage-valid cross-stage construction; exact split/concat; tree-complete select and slice; mean reward propagation plus group-normalized advantages; rejection of bad lineage, modality keys, and primitive batch sizes; packed variable-length text select/concat; and lazy tensor-reference selection, materialization, identity, and bounds behavior. The structured outcomes and data-sensitive SHA-256 round-trip hash are stored with the run artifacts.

Required before complete evidence admission. The subsequent reports describe E0 correctness closure, but its underlying test and configuration artifacts were not revalidated in this integration. The following system-level checks are the evidence requirements, not a claim that every experiment still lacks each check:

1. **Rollout–replay parity:** before the first update, compare engine-emitted and train-side replay log probabilities or transition statistics under stated numerical tolerances.
2. **Publication parity:** repeat the parity check after each LoRA or full-weight publication path used in an experiment.
3. **Batch-version integrity:** verify that an optimizer batch never mixes behavior versions when the configured algorithm assumes one version; this is meaningful only on the asynchronous and agentic paths, whose versions advance at publication.
4. **Boundary integrity:** checkpoint and evaluation begin only after the async manager is empty; resume restores the optimizer version and republishes it before new rollout.
5. **Work accounting:** compare generated tokens or pixels, denoising and replay steps, scored samples, and committed optimizer updates across systems.
6. **Structural preconditions:** validate unique root IDs, equal part counts, parent-major grouping, shared-field equality, and version provenance before merging or propagating credit; check against an ID-joined reference.